Health-Check Software für die Dienste der Firma

DIPLOMARBEIT

verfasst im Rahmen der

Reife- und Diplomprüfung

an der

Höheren Abteilung für IT-Medientechnik

Eingereicht von:
Elias Brandstätter
Zoe Emily Öllinger

Betreuer:
Prof. DI Christian Aberger

Projektpartner:
hello again GmbH

Leonding, 7. April 2026

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt bzw. die wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht habe.

Die Arbeit wurde bisher in gleicher oder ähnlicher Weise keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Die vorliegende Diplomarbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Leonding, 7. April 2026 Elias Brandstätter & Zoe Emily Öllinger

Abstract

This diploma thesis was made in cooperation with the austrian software company Hello Again. Our goal was to develop a Health Monitoring Dashboards that helps their employees to keep track of the health status of every Loyalty App. The dashboard shows if an app is available and healthy, which problems may occur in the Google Play Store or Apple App Store, and how many open tickets have been created regarding this app. On the technical side the dashboard was developed by using Vue.js in the frontend after we designed it with Figma, in the backend we used FastAPI with Uvicorn and Pydantic and is connected with extern services like Google Play Developer API, die App Store Connect API, HubSpot and Asana. We developed the dashboard in our own Git-Branch and in the end our code got transferred in the company's project.

Zusammenfassung

Die vorliegende Diplomarbeit wurde in Zusammenarbeit mit dem österreichischen Softwareunternehmen Hello Again erarbeitet. Ziel war die Entwicklung eines Health Monitoring Dashboards, das den Mirabeitern einen schnellen Überblick über den Zustand aller betreuten Kunden-Apps bietet. Das Dashboard zeigt auf einen Blick, ob eine Kunden-App verfügbar und gesund ist, welche Probleme im Google Play Store oder Apple App Store aufgetreten sind, und wie viele offene Tickets einer App zugeordnet sind. Technisch wurde das Frontend mit Vue.js umgesetzt, das zuvor in Figma designed wurde. Das Backend wurde mit FastAPI, mit Uvicorn und Pydantic umgesetzt und ist an mehrere externe Dienste angebunden, darunter die Google Play Developer API, die App Store Connect API, HubSpot und Asana. Die Entwicklung fand in einem eigenen Git-Branch statt und wurde am Ende in das offizielle Projekt von Hello Again eingeführt.

Inhaltsverzeichnis

1 Einleitung

1.1 Ausgangslage

Hello Again ist ein Unternehmen, das sich auf die Entwicklung von Kundenbindungs-Apps für kleine und mittelständische Unternehmen spezialisiert hat. Diese Apps werden über verschiedene Apple- und Google-Developer-Accounts veröffentlicht und gepflegt, wobei je nach Kunde und Plattform unterschiedliche Accounts und Konfigurationen zum Einsatz kommen. Da die Anzahl der betreuten Kunden-Apps kontinuierlich wächst, steigt damit auch der Aufwand, den Status und die Korrektheit all dieser Apps und der zugehörigen Accounts im Blick zu behalten.

1.2 Problemstellung

In der Praxis zeigt sich, dass Probleme mit einzelnen Kunden-Apps häufig nicht sofort erkannt werden. Typische Ursachen sind fehlende bzw. abgelaufene Verifizierungen oder fehlerhafte Konfigurationen in den Developer Accounts. Solche Probleme können dazu führen, dass eine App eingeschränkt verfügbar ist oder im schlimmsten Fall aus dem jeweiligen Store entfernt wird - mit direkten Konsequenzen für den betroffenen Kunden.

Da diese Informationen bislang über mehrere Plattformen verteilt sind und keine zentrale Übersicht existiert, ist eine systematische Überwachung aller Apps mit vertretbarem Aufwand kaum möglich. Probleme werden dadurch oft erst dann bemerkt, wenn sie bereits eskaliert sind und der Handlungsdruck hoch ist. Ein proaktiver, strukturierter Ansatz zur Fehlererkennung fehlt.

1.3 Ziel

Ziel dieses Projekts ist die Entwicklung eines zentralen Health-Monitoring-Dashboards, das den Status aller betreuten Kunden-Apps auf einen Blick erfassbar macht. Das

System aggregiert Informationen aus verschiedenen Quellen - darunter die Google Play Developer API, die App Store Connect API, HubSpot sowie Asana - und stellt diese in einem übersichtlichen Dashboard dar.

Jede App wird dabei automatisch einer von drei Statuskategorien zugeordnet: *Healthy*, *Needs Attention* oder *Critical*. Diese Kategorisierung ermöglicht es, den Handlungsbedarf auf einen Blick einzuschätzen und Prioritäten gezielt zu setzen. Darüber hinaus unterstützt das System den Fehlerbehebungsprozess aktiv, indem es strukturierte Hinweise auf die Ursache eines Problems liefert und direkte Maßnahmen vorschlägt.

Um den Arbeitsalltag weiter zu erleichtern, bietet das Dashboard umfangreiche Filter- und Suchmöglichkeiten, mit denen sich Apps nach bestimmten Fehlermeldungen, Parametern oder Plattformen eingrenzen lassen. Insgesamt soll das System dazu beitragen, Probleme frühzeitig zu erkennen, Reaktionszeiten zu verkürzen und die Qualität der betreuten Apps nachhaltig zu verbessern.

2 Umfeldanalyse

2.0.1 Hello Again

Hello Again ist ein österreichisches Softwareunternehmen, das sich auf digitale Kundenbindungslösungen spezialisiert hat. Das Kernprodukt ist eine Kundenbindungsapp, mit der Unternehmen ihren Kunden digitale Stempelkarten, Punkte und Prämien sowie Gewinnspiele anbieten können. Die App wird von Hello Again entwickelt und betrieben, aber im Namen der jeweiligen Kundenfirmen veröffentlicht.

Das Unternehmen betreut viele Kunden gleichzeitig. Die Apps sind sowohl im Google Play Store als auch im Apple App Store verfügbar. Um den Überblick über den Zustand aller Apps und Kundenaccounts zu behalten, wurde das Health Monitoring Dashboard entwickelt.

[**?**]

2.0.2 Projektteam und Zusammenarbeit

Wir waren als Praktikanten zu Hello Again gekommen und wurden den Devs vorgestellt. Zu Beginn des Projekts fand eine Einschulung statt, in der uns die wichtigsten Richtlinien und Tools erklärt wurden. Die Betreuung erfolgte durch einen fixen Ansprechpartner im Unternehmen, an den wir uns bei Fragen oder Problemen wenden konnten.

Wir arbeiteten in einem eigenen Git-Branch namens health-monitoring, der vom offiziellen Main-Branch getrennt war. Innerhalb dieses Branches haben wir uns zwei weitere Branches angelegt einen für das Frontend und einen für das Backend. Am Ende des Projekts wurde unsere Arbeit von den Entwicklern geprüft und in das offizielle Projekt übernommen.

2.0.3 Markt und Branche

Der Markt für Kundenbindungslösungen wächst seit einigen Jahren kontinuierlich. Immer mehr Unternehmen ersetzen ihre physischen Stempelkarten durch digitale Apps, wie zum Beispiel Müller, weil diese einfacher zu verwalten sind und gleichzeitig Daten über das Kaufverhalten der Kunden liefern. Mit Gamifikation kann man Kunden motivieren mehr zu kaufen, da sie täglich die App benutzen, wie zum Beispiel, um am Glücksrad Rabatte zu gewinnen. So öffnen die Kunden die Apps öfter und bekommen immer die neuesten Angebote angezeigt, dass den Kaufreiz erhöht und sie dazu bringt öfter bei der selben Firma einzukaufen.

[**?**]

3 Technologien

3.1 Overview

3.1.1 Techstack

Der Techstack des Projekts lässt sich in sechs Schichten einteilen, die vom Design bis zur Cloud reichen.

Design - Figma

Bevor gecoded wurde, wurden die Konzepte, das Design und die Userexperience in Figma erarbeitet. Im Projekt wurden damit sämtliche Ansichten des Health Monitoring Dashboards visuell erstellt, wie zum Beispiel die Layouts und das Farbschema sowie die Kommentaransicht und die Health-Checks.

Frontend - Vue.js

Die Benutzeroberfläche wurde mit Vue.js umgesetzt, einem JavaScript-Framework für die Entwicklung von Single Page Applications. Jedes UI-Element, sowie die Filtertabs, die Übersichtstabelle oder der Detail-Button wurden als eigenständige, wiederverwendbare Komponente programmiert.

Backend - FastAPI, Uvicorn, Pydantic, JavaScript

Das Backend bildet den wichtigsten Part der Datenverarbeitung. Als primäres API-Framework wurde FastAPI verwendet.

Externe APIs - Google Play, App Store Connect, HubSpot, Asana

Es wurden mehrere externe Dienste über ihre APIs eingebunden. Die Google Play Developer API und die App Store Connect API liefern Verfügbarkeits und Statusdaten. Asana dient als Projektmanagement-Tool und liefert die im Dashboard sichtbaren Ticket und Task Zahlen der einzelnen Clients. HubSpot wird als CRM-System eingebunden, um Kundenstammdaten abzurufen.

Datenhaltung - BigQuery, SQL

Für die Speicherung großer Datenmengen wird Google BigQuery verwendet. Die Abfragesprache SQL wird genutzt, um Datensätze wie MRR-Werte dem Backend zur Verfügung zu stellen.

Datenhaltung - BigQuery, SQL

Die Infrastruktur läuft auf der Google Cloud Platform. GCP übernimmt Hosting, Deployment und den Betrieb des Backends.

3.2 Frontend

3.2.1 Vue.js

Das Frontend des Health-Monitoring-Dashboards wurde mit dem JavaScript-Framework Vue.js implementiert. Die Entscheidung für Vue.js basiert auf seiner einfachen Integration, klaren Struktur sowie der Weiterentwicklung innerhalb der Firma. Da innerhalb der Firma für die Entwicklung des Dashboards bereits genutzt wird, bietet uns diese Technologie perfekte Vorraussetzungen um implementiert zu werden. Dabei bietet Vue.js uns auch eine komponentenbasierte Architektur, wodurch einzelne UI-Bestandteile unabhängig voneinander Entwickelt und wiederverwendet werden können. Hierbei wurden uns von der Firma bereits fertige Komponenten zur Verfügung gestellt, wie zum Beispiel die Pagination. Für uns sind die Komponenten von Vorteil, da wir im Dashboard Übersichtsseiten, Detailseiten und Statusanzeigen nutzen. Dabei muss man beachten, dass von cer Firma für das Frontend zwei Komponenten gewünscht waren. Einmal für die Übersicht und einmal für die Detailseite.

Ein weiterer Vorteil bei unserer Verwendung von Vue.js ist, die reaktive Datenbindung, da Änderungen an den Daten automatisch im User Interface Aktualisiert werden, wodurch eine dynamische und aktuelle Darstellung der Health-Daten gewährleitstet wird.

[?]

Framework Background

Vue.js wurde von Evan You entwickelt und erstmals im Jahr 2014 veröffentlicht. Hierbei verfolgt das Framework einen progressiven Ansatz, bei dem die Entwickler selbst entscheiden können, wie umfangreich man Vue in sein Projekt integriert.

Ein zentrales Konzept von Vue ist das Reactive Data Binding. Dabei wird das DOM automatisch aktualisiert, wenn sich Daten verändern.

Zusätzlich bietet Vue eine klare, strukturierte Trennung zwischen Template, Styling und Logik, wodurch die Wartbarkeit und Erweiterbarkeit der Anwendung besser verständlich ist.

Einsatz im Frontend

Im Rahmen dieses Projekts übernimmt Vue.js folgende zentrale Aufgaben:

- Darstellung der Health-Check-Daten
- Visualisierung von Statusinformationen und Fehlermeldungen
- Interaktion mit den APIS des Backends
- Dynamisches Laden und Aktualisieren von Daten
- Navigation zwischen verschiedenen Ansichten

[?]

3.2.2 UI&UX Design mit Figma

Für die Gestaltung der Benutzeroberfläche wurde das Design-Tool Figma verwendet. Figma ermöglicht eine kollaborative Erstellung von UI-Designs und bietet uns umfangreiche Funktionen zur Visualisierung unserer Benutzeroberflächen.

In unserem Projekt diente Figma als zentrales Werkzeug zur Planung uns Ausarbeitung des User Interfaces und des gewünschten Workflows. Dabei wurden von uns Layouts, Workflow und Farbkonzepte entwickelt, bevor diese dann technisch umgesetzt wurden.

[?]

Designprozess

Der Designprozess erfolgte iterativ und war stark durch regelmäßige Feedback-Stunden geprägt. In mehreren Workflow und Feedback-Sessions wurden die Entwürfe gemeinsam besprochen und anschließend verbessert und an die Anforderungen der Mitarbeiter angepasst.

Diese Vorgehensweise ermöglichte es uns, frühzeitig Verbesserungen zu erkennen und entsprechend darauf zu reagieren und die Benutzerfreundlichkeit auf Wunsch der Mitarbeiter kontinuierlich zu optimieren.

[?]

Vorteile von Figma

Der Einsatz von Figma bringt mehrere Vorteile mit sich:

- Zentrale und kollaborative Designplattform
- Schnelle Erstellung von Prototypen
- Einfache Abstimmung im Team
- Möglichkeit zur direkten Visualisierung von User Flows
- Konsistente Gestaltung durch wiederverwendbare Komponenten

[?]

3.2.3 Benutzerfreundlichkeit und Interaktion

Ein zentrales Ziel des Frontends war die Entwicklung einer intuitiven und übersichtlichen Benutzeroberflache.

Besonderes Augenmerk wurde auf folgende Aspekte gelegt:

- Übersichtliche Darstellung von Statusinformationen
- Klare visuelle Hervorhebung von Fehlern und Warnungen
- Schnelle Navigation zwischen verschiedenen Bereichen
- Minimierung von Ladezeiten durch effiziente Datenabfragen
- Kommentarsektion zur Plattform-Übergreifenden bearbeitung
- Anzeige von gravierenden Fehlern mit Verlinkung zur Fehlerbehebung und Dokumentation

3.3 Backend

3.3.1 Python

Das Backend des Health-Monitoring-Systems wurde mit der Programmiersprache Python implementiert. Die Entscheidung für Python war dabei aus mehreren Gründen naheliegend: Einerseits entspricht Python den internen Entwicklungsrichtlinien des Unternehmens, andererseits zeichnet sich die Sprache durch eine hohe Lesbarkeit und klare Syntax aus, die es ermöglicht, auch komplexe Logik übersichtlich zu strukturieren.

Darüber hinaus verfügt Python über ein umfangreiches Ökosystem an Bibliotheken für Webentwicklung, Datenanalyse und Cloud-Integration. Gerade im Kontext dieses Projekts ist die Verfügbarkeit offizieller Clientbibliotheken für Google BigQuery von besonderer Bedeutung, da diese die Anbindung an die Datenbankinfrastruktur erheblich vereinfachen. Nicht zuletzt profitiert das Projekt von der starken Community rund um Python, die umfangreiche Dokumentationen und etablierte Best Practices bereitstellt - beides Faktoren, die die langfristige Wartbarkeit des Systems begünstigen.

Hintergrund der Sprache

Python ist eine interpretierte, objektorientierte Programmiersprache, die ursprünglich von Guido van Rossum entwickelt und erstmals 1991 veröffentlicht wurde. Das erklärte Ziel bei ihrer Entwicklung war, eine Sprache zu schaffen, die gleichermaßen einfach zu lesen und effizient zu verwenden ist. Erreicht wird das durch eine minimalistische Syntax, die sich stark an natürlicher Sprache orientiert und dadurch die Lesbarkeit und Nachvollziehbarkeit von Code deutlich verbessert.

Im Gegensatz zu kompilierten Sprachen wird Python interpretiert ausgeführt, was die Entwicklung beschleunigt, da Änderungen am Code unmittelbar getestet werden können. Für die Entwicklung von Web-Backends und Datenverarbeitungssystemen ist diese Eigenschaft besonders vorteilhaft. [**?**, **?**]

Einsatz im Backend

Im Rahmen dieses Projekts übernimmt Python mehrere zentrale Aufgaben innerhalb der Backend-Architektur: die Implementierung der REST-API, die Kommunikation mit externen Schnittstellen, die Verarbeitung und Aggregation von Monitoring-Daten, Datenbankabfragen über BigQuery sowie die Logik zur Statusanalyse und Fehlererkennung.

Besonders hervorzuheben ist dabei die Fähigkeit, verschiedene externe Systeme über API-Schnittstellen anzubinden. Im Health-Monitoring-System werden Daten aus mehreren unterschiedlichen Plattformen verarbeitet - darunter App-Store-Systeme, CRM-Systeme und Projektmanagement-Tools. Python ermöglicht es, diese heterogenen Datenquellen in einer zentralen Backend-Logik zusammenzuführen und strukturiert an das Frontend weiterzugeben.

3.3.2 FastAPI

Für die Implementierung der Backend-Schnittstellen wurde das Python-Framework FastAPI eingesetzt. FastAPI ist ein modernes, leistungsfähiges Framework zur Entwicklung von Web-APIs, das auf dem ASGI-Standard (Asynchronous Server Gateway Interface) basiert und eine schnelle sowie strukturierte Entwicklung von REST-Schnittstellen ermöglicht. Im Health-Monitoring-System stellt FastAPI die Endpunkte bereit, über die das Frontend Statusinformationen zu den überwachten Apps abruft und externe APIs angebunden werden.

Hintergrund des Frameworks

FastAPI wurde von Sebastián Ramírez entwickelt und 2018 erstmals veröffentlicht. Das Framework macht konsequenten Gebrauch von modernen Python-Features - insbesondere Typannotationen und asynchroner Programmierung - und hebt sich dadurch von älteren Frameworks wie Flask oder Django REST Framework ab.

Ein zentrales Merkmal von FastAPI ist die Nutzung von Python-Typdefinitionen zur automatischen Validierung von Anfragen und Antworten. Request-Parameter und Response-Objekte können dadurch klar definiert werden, was die Stabilität und Vorhersehbarkeit der API erhöht. Zusätzlich generiert FastAPI automatisch eine OpenAPI-Spezifikation samt interaktiver Dokumentation, die Entwicklern direkt im Browser zur Verfügung steht und das Testen einzelner Endpunkte erheblich erleichtert. [**?**]

REST-API-Architektur

Das Backend des Health-Monitoring-Systems folgt einer klassischen REST-Architektur, bei der das System verschiedene Endpunkte bereitstellt, über die Daten abgefragt oder Aktionen ausgelöst werden können. Dazu zählen etwa das Abrufen aller Health-Checks, das Anzeigen von Details zu einem bestimmten Kunden, das Erstellen von Tasks oder Kommentaren sowie das Abfragen von Ticket-Informationen. Diese Endpunkte werden in FastAPI über Route-Deklarationen definiert, wobei jede Route einer spezifischen HTTP-Methode - etwa GET oder POST - zugeordnet ist.

Die REST-Architektur ermöglicht eine klare Trennung zwischen Frontend und Backend, sodass beide Systeme unabhängig voneinander weiterentwickelt werden können, solange die vereinbarte Schnittstellendefinition eingehalten wird.

Performance und Skalierbarkeit

FastAPI gehört zu den performantesten Python-Webframeworks, was sich vor allem auf die native Unterstützung asynchroner Programmierung zurückführen lässt. [**?**] Im Kontext dieses Projekts ist das besonders relevant, da im Zuge einer einzelnen Anfrage häufig mehrere externe APIs parallel abgefragt werden müssen. Durch die asynchrone Verarbeitung können diese Anfragen gleichzeitig ausgeführt werden, was die Gesamtantwortzeit spürbar reduziert und das System auch bei höherer Last stabil hält.

3.3.3 Uvicorn

Zur Ausführung der FastAPI-Anwendung wird der ASGI-Server Uvicorn eingesetzt. Uvicorn ist ein leichtgewichtiger, performanter Webserver für Python-Anwendungen und übernimmt die Aufgabe, eingehende HTTP-Requests entgegenzunehmen und an die FastAPI-Anwendung weiterzuleiten. Nach der Verarbeitung durch das Framework wird die generierte Response über Uvicorn an den Client zurückgesendet. [**?**]

Der ASGI-Standard

Uvicorn basiert auf dem ASGI-Standard (Asynchronous Server Gateway Interface), der eine moderne Alternative zum älteren WSGI-Standard darstellt. Der wesentliche Unterschied liegt in der Unterstützung asynchroner Verarbeitung: Während bei klassischen synchronen Webservern jede eingehende Anfrage einen eigenen Thread blockiert, können ASGI-basierte Server mehrere Operationen gleichzeitig verwalten, ohne dabei auf den Abschluss einer einzelnen Anfrage warten zu müssen. Das ist besonders vorteilhaft für Anwendungen, die häufig externe APIs aufrufen oder auf Datenbankabfragen warten - beides trifft auf das Health-Monitoring-System zu. [**?**]

Einsatz im Projekt

Im Health-Monitoring-System fungiert Uvicorn als Laufzeitumgebung für die FastAPI-Anwendung. Der Server startet die API, stellt sie über einen definierten HTTP-Port zur Verfügung und verwaltet eingehende Anfragen, indem er diese an FastAPI weiterleitet und die Antworten zurückgibt. Durch die hohe Performance von Uvicorn kann das

Backend auch bei mehreren gleichzeitigen Anfragen effizient arbeiten, ohne dass es zu merklichen Verzögerungen kommt.

3.3.4 Pydantic

Für die Datenvalidierung und strukturierte Modellierung von Datenstrukturen wird die Python-Bibliothek Pydantic ausgewählt. Pydantic ermöglicht die Definition von Datenmodellen, die beim Empfang oder Versand von Daten automatisch validiert und bei Bedarf konvertiert werden. Im Health-Monitoring-System kommen diese Modelle vor allem bei der Verarbeitung von API-Requests und Responses zum Einsatz, wo sie sicherstellen, dass alle übermittelten Daten dem erwarteten Format entsprechen. [**?**]

Automatische Datenvalidierung

Ein zentrales Merkmal von Pydantic ist die automatische Validierung eingehender Daten. Sobald ein Client eine Anfrage an die API sendet, prüft Pydantic, ob alle erforderlichen Felder vorhanden sind und ob diese den definierten Datentypen entsprechen. Wird etwa erwartet, dass ein bestimmtes Feld eine Zeichenkette enthält oder ein Datum in einem vorgegebenen Format übergeben wird, erkennt Pydantic Abweichungen davon sofort und gibt eine entsprechende Fehlermeldung zurück. Fehlerhafte Eingaben werden damit bereits an der Systemgrenze abgefangen, bevor sie in die eigentliche Verarbeitungslogik gelangen. [**?**]

Datenmodelle im System

Pydantic verwendet sogenannte Model-Klassen, um die Struktur von JSON-Objekten zu beschreiben, die über die API übertragen werden. Im Health-Monitoring-System werden auf diese Weise Modelle für Health-Check-Ergebnisse, Kundeninformationen, Task-Objekte und Ticket-Daten definiert. Diese einheitliche Strukturierung stellt sicher, dass Daten innerhalb des gesamten Systems konsistent repräsentiert werden und keine unerwarteten Formate in der Verarbeitungslogik auftreten.

Integration mit FastAPI

Ein besonderer Vorteil von Pydantic im Kontext dieses Projekts ist die native Integration mit FastAPI. Da FastAPI Pydantic-Modelle direkt für die Definition von Request- und Response-Schemata verwendet, entsteht eine durchgehend typsichere API, bei der Validierung, Dokumentation und Schnittstellendefinition aus einer gemeinsamen Modellbeschreibung abgeleitet werden. Das reduziert redundanten Code und trägt zur Konsistenz der gesamten Backend-Architektur bei.

3.4 Cloud & Datenerhaltung

3.4.1 Google BigQuery

Für die Speicherung und Analyse der Monitoring-Daten wird Google BigQuery eingesetzt - ein von Google verwaltetes Data-Warehouse-System, das speziell für die Verarbeitung großer Datenmengen ausgelegt ist. Da das Gesamtsystem ohnehin auf der Google Cloud Platform betrieben wird, fügt sich BigQuery nahtlos in die bestehende Infrastruktur ein und war daher eine logische Wahl für diesen Einsatzzweck.

Im Health-Monitoring-System dient BigQuery als zentrale Datenquelle für die Statusinformationen aller überwachten Kunden-Apps. Die anfallenden Monitoring-Daten werden dort regelmäßig aggregiert und können anschließend per SQL-Abfrage vom Backend abgerufen werden. Ein wesentlicher Vorteil dabei ist die serverlose Architektur von BigQuery: Skalierung, Speicherverwaltung und Performanceoptimierung werden vollständig von der Plattform übernommen, sodass kein Aufwand für den Betrieb einer eigenen Datenbankinfrastruktur entsteht. [**?**]

Architektur und Funktionsweise

Die Eignung von BigQuery für analytische Workloads lässt sich auf die zugrunde liegende Architektur zurückführen. BigQuery verwendet eine sogenannte Columnar Storage Architektur, bei der Daten spaltenweise statt zeilenweise gespeichert werden. [**?**] Für analytische Abfragen ist das ein entscheidender Vorteil: Da typischerweise nur ein Teil der vorhandenen Spalten benötigt wird, müssen nicht alle gespeicherten Daten gelesen werden, was Abfragen erheblich beschleunigt und gleichzeitig eine effizientere Datenkompression ermöglicht. Darüber hinaus verteilt BigQuery die Verarbeitung von

Abfragen automatisch auf viele parallele Rechenknoten, sodass auch komplexe Analysen über sehr große Datensätze in kurzer Zeit abgeschlossen werden können.

Einsatz im Monitoring-System

Im Health-Monitoring-System werden mehrere BigQuery-Tabellen genutzt, die unterschiedliche Aspekte des Systems abdecken: Health-Check-Ergebnisse der Apps, HubSpot-Ticket-Daten, App-Konfigurationsinformationen sowie Monitoring-Daten aus dem App Store und Google Play. Diese Aufteilung in thematisch getrennte Tabellen ermöglicht es, Daten gezielt abzufragen, ohne bei jeder Anfrage unnötige Informationen laden zu müssen.

Der Zugriff auf BigQuery erfolgt im Backend über die offizielle Python-Clientbibliothek. Die SQL-Abfragen werden dabei dynamisch zur Laufzeit generiert und ausgeführt, abhängig davon, welche App-Daten jeweils benötigt werden. Die zurückgelieferten Ergebnisse werden anschließend im Backend aufbereitet und als strukturierte Daten an das Frontend-Dashboard weitergegeben.

Vorteile für das Projekt

BigQuery erweist sich für dieses Projekt aus mehreren Gründen als geeignete Wahl. Die serverlose Architektur reduziert den Betriebsaufwand erheblich, da keine Infrastruktur manuell verwaltet werden muss. Analytische SQL-Abfragen können auch bei wachsenden Datenmengen performant ausgeführt werden, ohne dass Anpassungen am System erforderlich werden. Nicht zuletzt vereinfacht die enge Integration in die Google Cloud Platform die Verwaltung von Authentifizierung und Zugriffsrechten, da diese zentral über die GCP-Infrastruktur abgewickelt werden.

3.4.2 Google Cloud Platform

Die Infrastruktur des Health-Monitoring-Systems basiert auf der Google Cloud Platform (GCP), Googles umfassender Cloud-Computing-Plattform. GCP stellt eine breite Palette an Diensten bereit - von Hosting und Datenspeicherung über Datenverarbeitung bis hin zu KI-Funktionalitäten - und bildet das gemeinsame Fundament, auf dem die verschiedenen Systemkomponenten betrieben und miteinander verbunden werden. [?]

Cloud Computing als Grundlage

Die Entscheidung, das System auf einer Cloud-Plattform aufzubauen, folgt einem in der modernen Softwareentwicklung weit verbreiteten Ansatz: Anstatt eigene Server zu betreiben, werden IT-Ressourcen als Service über das Internet bezogen. [**?**] Für ein Projekt dieser Art bietet das konkrete Vorteile - insbesondere, weil der Entwicklungsaufwand so auf die eigentliche Anwendungslogik konzentriert werden kann, ohne dass Infrastrukturthemen im Vordergrund stehen. Darüber hinaus ermöglicht die Cloud eine hohe Verfügbarkeit und lässt sich bei steigendem Bedarf ohne zusätzliche Hardwareanschaffungen skalieren.

Integration im System

Im Rahmen des Projekts wird GCP vor allem für drei Zwecke genutzt: den Zugriff auf BigQuery als zentrale Datenquelle, die Authentifizierung der Backend-Komponenten über Service Accounts sowie die Verwaltung von API-Zugriffsrechten zwischen den einzelnen Diensten. Das Konzept der Service Accounts spielt dabei eine zentrale Rolle: Anstatt Zugangsdaten direkt im Code zu hinterlegen, erhält das Backend über einen dedizierten Service Account genau jene Berechtigungen, die für den Betrieb notwendig sind. Dieses Vorgehen folgt dem Prinzip der minimalen Rechtevergabe und trägt sowohl zur Sicherheit als auch zur Wartbarkeit des Systems bei.

Sicherheit und Skalierbarkeit

Ein weiterer Aspekt, der für den Einsatz von GCP spricht, ist die integrierte Sicherheitsarchitektur der Plattform. Über das IAM-System (Identity and Access Management) können Zugriffsrechte granular und rollenbasiert vergeben werden, sodass jede Systemkomponente ausschließlich auf die Ressourcen zugreifen kann, für die eine entsprechende Berechtigung vorliegt. Ergänzt wird dies durch verschlüsselte Datenübertragung sowie integrierte Monitoring- und Logging-Funktionen, die einen kontinuierlichen Einblick in den Systemzustand ermöglichen. Diese Eigenschaften erleichtern nicht nur den sicheren Betrieb, sondern auch die spätere Erweiterung des Systems, da neue Komponenten ohne großen Aufwand in die bestehende Infrastruktur eingebunden werden können.

3.4.3 SQL

Für den Zugriff auf die in BigQuery gespeicherten Monitoring-Daten wird SQL (Structured Query Language) verwendet. SQL ist die etablierte Standardsprache für relationale Datenbanken und ermöglicht es, Daten gezielt abzufragen, zu filtern, zu aggregieren und miteinander zu verknüpfen. [**?**, **?**] Da BigQuery SQL nativ unterstützt und die Sprache ausdrucksstark genug ist, um die im Monitoring-System benötigten Auswertungen präzise umzusetzen, war ihr Einsatz in diesem Kontext naheliegend.

Grundlagen von SQL

SQL wurde in den 1970er Jahren entwickelt und hat sich seither als universeller Standard für die Arbeit mit strukturierten Daten etabliert. Die Sprache umfasst Befehle für alle grundlegenden Datenbankoperationen - SELECT zum Abfragen, INSERT zum Einfügen, UPDATE zum Ändern und DELETE zum Entfernen von Datensätzen. Im analytischen Umfeld steht dabei der SELECT-Befehl klar im Vordergrund, da dort primär Daten ausgewertet und zusammengefasst werden, ohne den Datenbestand selbst zu verändern.

Einsatz im Projekt

Im Monitoring-System werden SQL-Abfragen hauptsächlich dazu verwendet, Health-Check-Daten und Ticket-Informationen aus BigQuery abzurufen, nach bestimmten Kunden-Apps zu filtern und die Ergebnisse sinnvoll zu gruppieren. Da sich die abgefragten Apps und Zeiträume je nach Anfrage unterscheiden, werden die Abfragen im Backend dynamisch zur Laufzeit generiert und ausgeführt. Die zurückgelieferten Ergebnisse werden anschließend in JSON-Strukturen überführt, die vom Frontend direkt verarbeitet werden können.

Vorteile für die Datenanalyse

SQL eignet sich für diesen Anwendungsfall besonders gut, weil es eine sehr präzise Kontrolle darüber ermöglicht, welche Daten in welcher Form abgerufen werden. Komplexe Filterungen, Aggregationen und Verknüpfungen lassen sich direkt in der Abfrage ausdrücken, ohne dass dafür umfangreiche Verarbeitungslogik im Backend implementiert werden müsste. In Kombination mit BigQuery können auf diese Weise auch größere

Datensätze effizient ausgewertet werden - die rechenintensive Verarbeitung übernimmt BigQuery, während sich das Backend auf die Weiterverarbeitung und strukturierte Aufbereitung der Ergebnisse konzentriert.

3.5 Externe Schnittstellen

3.5.1 Asana API

Zur Integration von Aufgabenmanagement-Daten wird die API der Projektmanagementplattform Asana verwendet. Über diese Schnittstelle können Tasks programmatisch erstellt, abgefragt und aktualisiert werden, ohne dass ein manueller Eingriff in die Plattform selbst erforderlich ist. [**?**]

Im Health-Monitoring-System wird die Asana API konkret dazu genutzt, offene Aufgaben zu bestimmten Kunden-Apps abzurufen und im Dashboard darzustellen. Das ermöglicht es, auf einen Blick zu erkennen, ob zu einem erkannten Problem bereits Maßnahmen eingeleitet wurden - eine Information, die für die Priorisierung von Reaktionen im Support-Prozess relevant ist.

API-Funktionalität

Die Asana API bietet Zugriff auf die zentralen Elemente des Projektmanagementsystems, darunter Projekte, Tasks, Kommentare, Benutzer und benutzerdefinierte Felder. Für das Monitoring-System sind dabei vor allem Tasks und deren Status von Bedeutung, da diese den aktuellen Bearbeitungsstand zu einem Problem widerspiegeln.

Integration im Monitoring-System

Die Kommunikation mit der Asana API erfolgt im Backend über HTTP-Anfragen. Dabei werden sowohl bestehende Aufgaben abgerufen als auch bei Bedarf neue erstellt. Jede Aufgabe enthält einen Titel, eine Beschreibung, eine Zuordnung zu einem Kunden-Slug sowie einen Verantwortlichen. Diese Informationen werden vom Backend verarbeitet und anschließend strukturiert im Monitoring-Dashboard dargestellt, sodass der Zusammenhang zwischen einer App und den zugehörigen offenen Aufgaben direkt ersichtlich ist.

3.5.2 HubSpot API

Neben den Aufgabendaten aus Asana werden über die HubSpot API auch Support- und CRM-Daten in das Monitoring-System integriert. HubSpot wird im Unternehmen zur Verwaltung von Kundenanfragen und Support-Tickets eingesetzt, und diese Informationen sind für ein vollständiges Bild des App-Status ebenso relevant wie technische Monitoring-Daten. [**?**]

CRM-Integration

CRM-Systeme (Customer Relationship Management) dienen der zentralen Verwaltung von Kundenbeziehungen und Supportanfragen. Durch die Einbindung dieser Daten in das Monitoring-System entsteht ein umfassenderer Überblick über den Zustand einer Kunden-App: Technische Auffälligkeiten lassen sich so direkt mit bereits bekannten Supportfällen in Verbindung bringen, was die Einschätzung der Lage erheblich erleichtert.

Verwendung im Dashboard

Konkret wird die HubSpot API dazu verwendet, offene Support-Tickets zu zählen und bestimmten Kunden zuzuordnen. Der daraus resultierende Supportstatus wird im Dashboard visualisiert und gibt dem Nutzer unmittelbar Auskunft darüber, ob zu einer App bereits aktive Supportvorgänge existieren. Das verhindert Doppelarbeit und schafft eine gemeinsame Informationsbasis für technische und supportverantwortliche Teams.

3.5.3 Google Play Developer API

Zur Überwachung des Status von Android-Anwendungen wird die Google Play Developer API eingebunden. Diese Schnittstelle ermöglicht den programmatischen Zugriff auf Informationen aus dem Google Play Entwicklerkonto und bildet damit die Grundlage für die automatisierte Überprüfung von Android-Apps im Monitoring-System. [**?**]

Funktionen der API

Die Google Play Developer API stellt unter anderem Informationen zum Veröffentlichungsstatus einer App, zu aktiven App-Versionen sowie zu etwaigen Richtlinienverstößen bereit. Gerade letztere sind für ein Monitoring-System von besonderer Bedeutung, da Richtlinienverstöße im schlimmsten Fall zur Delistung einer App führen können und daher frühzeitig erkannt werden sollten.

Bedeutung für das Monitoring

Im Health-Monitoring-System wird diese API genutzt, um den aktuellen Status der überwachten Android-Apps kontinuierlich zu prüfen. Probleme wie fehlende Verifizierungen, Einschränkungen der App-Verfügbarkeit oder aktive Richtlinienverstöße werden dabei automatisch erkannt und im Dashboard sichtbar gemacht, sodass zeitnah reagiert werden kann.

3.5.4 App Store Connect API

Analog zur Google Play Developer API wird für iOS-Anwendungen die App Store Connect API verwendet. Diese Schnittstelle bietet Entwicklern programmatischen Zugriff auf Informationen aus dem Apple Developer Account und ermöglicht damit eine automatisierte Überwachung des Status von iOS-Apps. [**?**]

API-Funktionalität

Über die App Store Connect API lassen sich App-Statusinformationen, der Verfügbarkeitsstatus im App Store, App-Metadaten sowie Account-Konfigurationen abrufen. Diese Daten geben Aufschluss darüber, ob eine iOS-App korrekt konfiguriert ist und ohne Einschränkungen im App Store verfügbar ist.

Einsatz im Monitoring-System

Im Monitoring-System wird diese API genutzt, um potenzielle Probleme mit iOS-Apps frühzeitig zu identifizieren. Dazu zählen etwa fehlende App-Store-Verfügbarkeit, Konfigurationsprobleme im Developer Account oder Einschränkungen durch Apple-Richtlinien.

Da diese Prüfungen automatisiert im Hintergrund ablaufen, werden entsprechende Auffälligkeiten unmittelbar im Dashboard visualisiert, ohne dass eine manuelle Kontrolle des Developer Accounts erforderlich ist.

4 Umsetzung

4.1 Systemarchitektur

Die Systemarchitektur des Health-Monitoring-Systems folgt einer mehrschichtigen Architektur, die aus einem Frontend, einem Backend sowie mehreren externen Datenquellen besteht. Ziel dieser Architektur ist es, Daten aus verschiedenen Plattformen zentral zu sammeln, zu analysieren und anschließend in einem übersichtlichen Dashboard darzustellen.

Durch die klare Trennung der einzelnen Komponenten wird eine modulare und wartbare Systemstruktur erreicht. Jede Schicht des Systems erfüllt eine spezifische Aufgabe und kommuniziert über definierte Schnittstellen mit den anderen Komponenten.

4.1.1 Gesamtübersicht der Architektur

Das System besteht aus vier zentralen Ebenen:

- Frontend (Dashboard)
- Backend (API und Datenverarbeitung)
- Cloud- und Datenebene
- Externe Schnittstellen

Das Frontend dient als Benutzeroberfläche für Mitarbeiter des Unternehmens und ermöglicht die Anzeige sowie Analyse des Status aller Kunden-Apps. Das Backend stellt eine REST-API bereit, über die das Frontend Daten abrufen kann. Gleichzeitig übernimmt das Backend die Verarbeitung und Aggregation der Daten aus verschiedenen Quellen.

Die Cloud- und Datenebene stellt die zentrale Datenbasis dar, während externe Schnittstellen zusätzliche Informationen aus Drittplattformen liefern.

4.1.2 Frontend-Schicht

Das Frontend stellt die Benutzeroberfläche für die Mitarbeiter dar. Hier wird die Health-Übersicht und die Detailseite der einzelenen Apps dargestellt.

Das Frontend besteht aus:

- UI & UX
- Health-Übersicht
- Detailseite
- Kommentar-Sektion
- Erkennung von Statusänderungen

4.1.3 Backend-Schicht

Das Backend stellt die zentrale Logik des Systems dar und fungiert als Vermittler zwischen Frontend, Datenbank und externen Schnittstellen. Die Backend-Komponente wurde mit FastAPI entwickelt und läuft auf dem ASGI-Server Uvicorn.

Die wichtigsten Aufgaben des Backends sind:

- Bereitstellung einer REST-API für das Frontend
- Abrufen und Verarbeiten von Daten aus Google BigQuery
- Integration externer APIs
- Durchführung von Statusanalysen
- Erkennung von Statusänderungen

Das Backend aggregiert Daten aus mehreren Quellen und stellt sie in strukturierter Form bereit. Diese Daten werden anschließend vom Frontend dargestellt.

Ein wichtiger Bestandteil der Backend-Logik ist die Analyse der Statuswerte der einzelnen Apps. Hierbei werden verschiedene Health-Checks durchgeführt und deren Ergebnisse miteinander kombiniert, um einen Gesamtstatus zu bestimmen.

4.1.4 Daten- und Cloud-Schicht

Die Datenbasis des Systems liegt in Google BigQuery, wo alle relevanten Monitoring-Daten gespeichert sind - darunter das aggregierte Health-Check-Label der jeweiligen Kunden-Apps, synchronisierte HubSpot-Ticket-Daten sowie App-Konfigurationsinformationen.

Das Backend greift auf diese Daten ausschließlich über SQL-Abfragen zu, was eine präzise Datenextraktion ohne unnötigen Overhead ermöglicht.

Ergänzend dazu speichert das Backend täglich einen lokalen Snapshot des aktuellen Systemzustands als CSV-Datei. Diese Snapshots dienen als Vergleichsbasis für die automatische Erkennung von Statusänderungen, ohne dass eine eigene Historiendatenbank betrieben werden muss.

4.1.5 Integration externer Systeme

Neben den eigenen Cloud-Ressourcen bindet das System zwei externe Plattformen ein, deren Daten das Bild über den Zustand einer Kunden-App vervollständigen. Asana liefert Informationen über offene Aufgaben, die einem Kunden zugeordnet sind, und ermöglicht das Erstellen neuer Tasks direkt aus dem Dashboard heraus. HubSpot stellt Support-Ticket-Daten bereit, die Aufschluss darüber geben, ob bereits aktive Supportfälle zu einer App existieren. Beide Datenquellen werden im Backend mit den BigQuery-Daten zusammengeführt und in einer einheitlichen Struktur an das Frontend übergeben.

4.1.6 Datenfluss im System

Der typische Datenfluss lässt sich anhand des Aufrufs der Health-Check-Übersicht nachvollziehen:

1. Das Frontend sendet eine HTTP-Anfrage an den `/health-checks`-Endpunkt, optional ergänzt durch Filterparameter.
2. Das Backend führt an die BigQuery-Tabelle `health_monitoring.health_data` eine SQL-Abfrage aus und lädt parallel die aktuellen HubSpot-Ticketanzahlen sowie offene Asana-Tasks.
3. Die Daten aus allen Quellen werden pro Kunden-Slug zusammengeführt und mit dem zuletzt gespeicherten Snapshot verglichen, um Statusänderungen zu erkennen.
4. Das Backend gibt die aufbereiteten Daten als strukturiertes JSON-Objekt an das Frontend zurück, das die Ergebnisse im Dashboard darstellt.

4.2 UI und UX

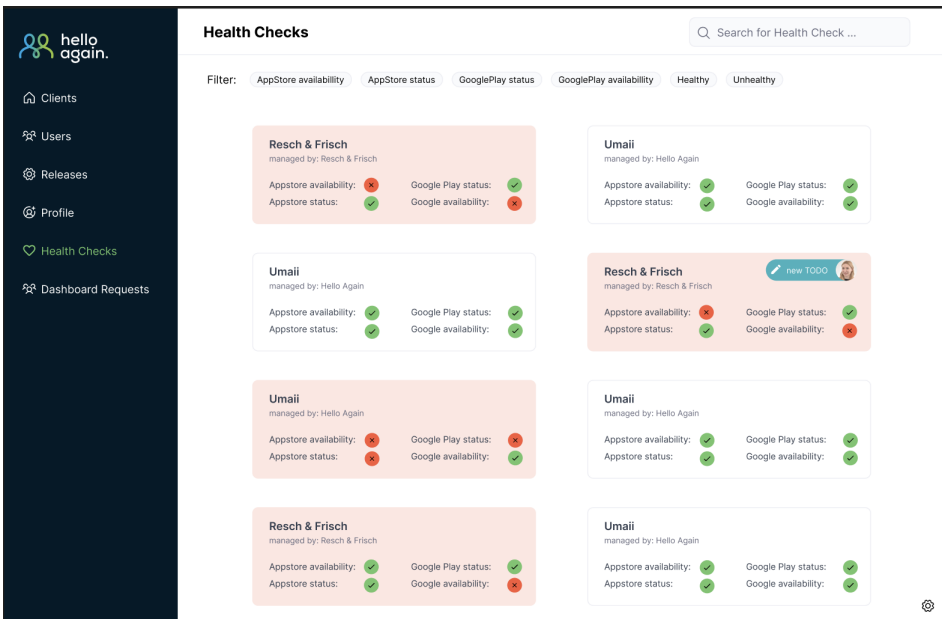
4.2.1 UI und UX Design

Das Design des Frontends wurde zu Beginn in Figma ausgearbeitet und weiters durch absprache und Feedback-Runden verbessert.

Iteration 1:

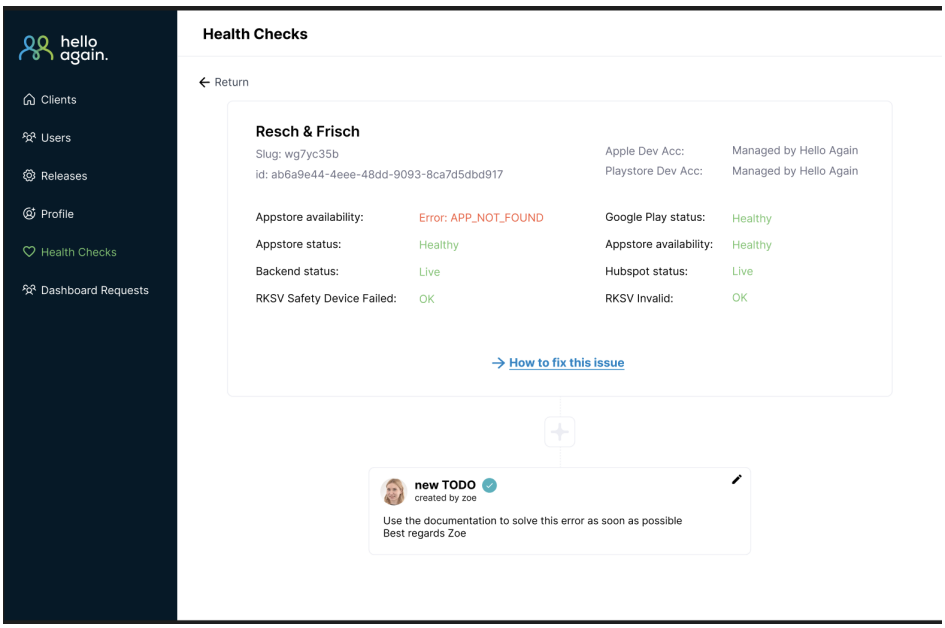
Im ersten Meeting mit der Firma wurden uns die Anforderungen für unsere Anwendung dargelegt und es wurde gebeten, zwei verschiedene Workflows zu entwickeln. Dabei benötigen wir eine Übersichtsseite und eine Detailseite sowie einen Weg um zur Dokumentation in Confluence zu kommen und weiters eventuell auch Statistiken.

Workflow-Version 1



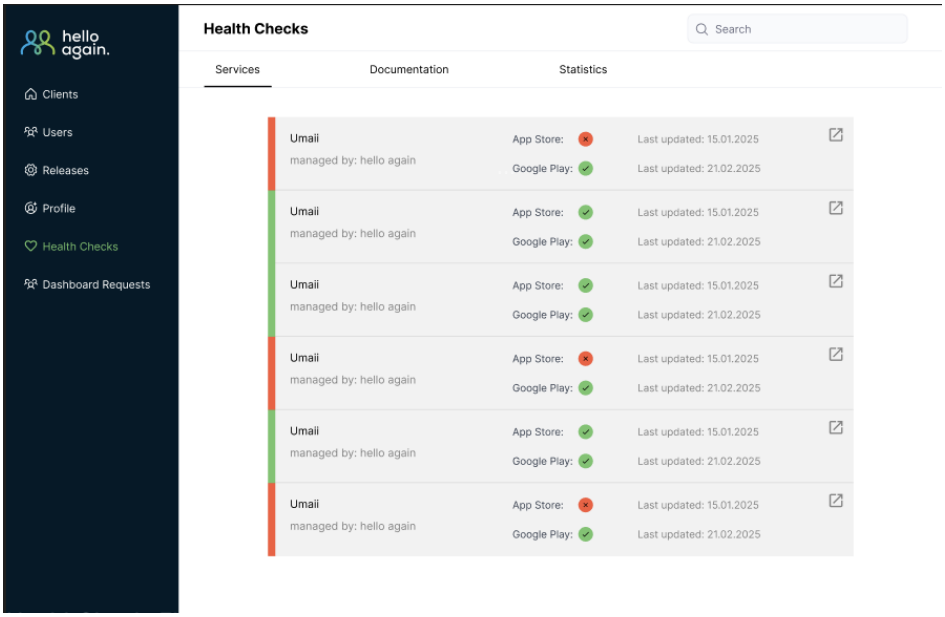
Dies ist die erste Version der Übersichtsseite. Hier werden die verschiedenen Kunden angezeigt, sowie ein grober Überblick über die Health-Stati. Je nach Farbe wird angezeigt ob die jeweiligen Apps Healthy oder Unhealthy sind (also ob sie Error werfen oder nicht). Sobald eine App ausfällt wird sie sofort gekennzeichnet, damit die Mitarbeiter auch dementsprechend auf diese Ausfälle reagieren können.

Dies ist die erste Version der Detailseite, auf der nochmal genauere Daten der Kunden angezeigt werden, sowie den Health-Status der einzelnen Health-Checks. Wird ein Fehler



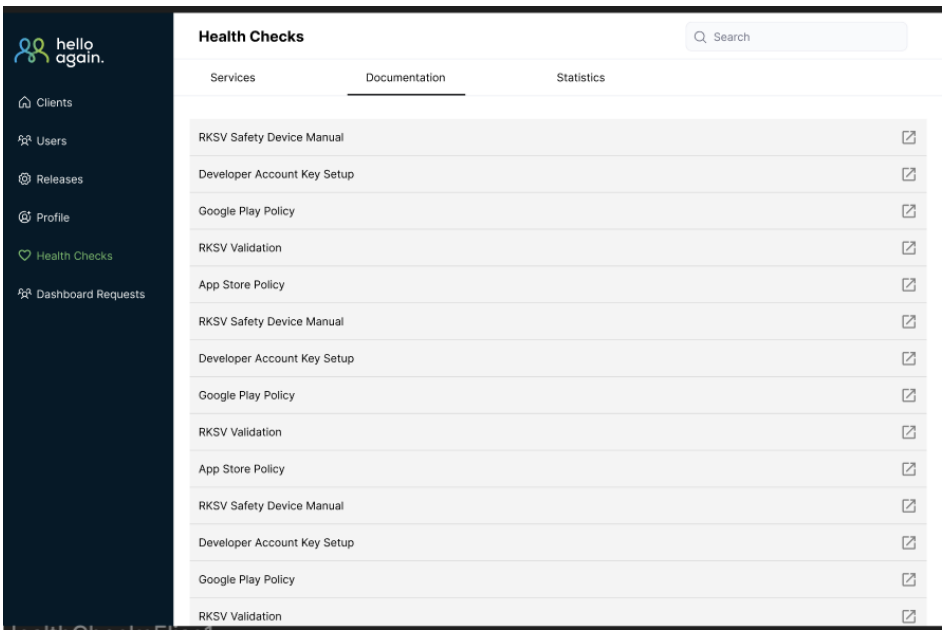
gemeldet wird auf der Detailsseite auch automatisch der Link angezeigt zur Lösung des Errors. Dies ermöglicht den Mitarbeitern eine effiziente und schnelle Arbeitsweise. Weiter unsen sehen wir noch eine TODO-Funktion. Hier kann man direkt im Dashboard Kommentare schreiben, die dann Plattformübergreifend in Asana erstellt werden.

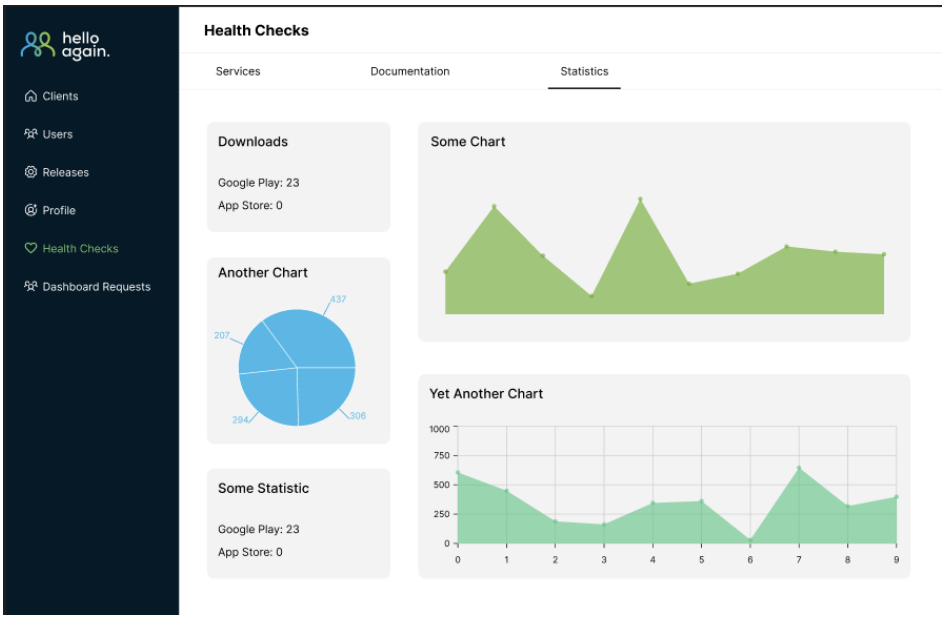
Workflow-Version 2



Dies ist die zweite Workflow-Version die erarbeitet wurde. Hier haben wir ebenfalls eine Übersichtsseite mit einer Liste doch mit einem kleinen Unterschied.

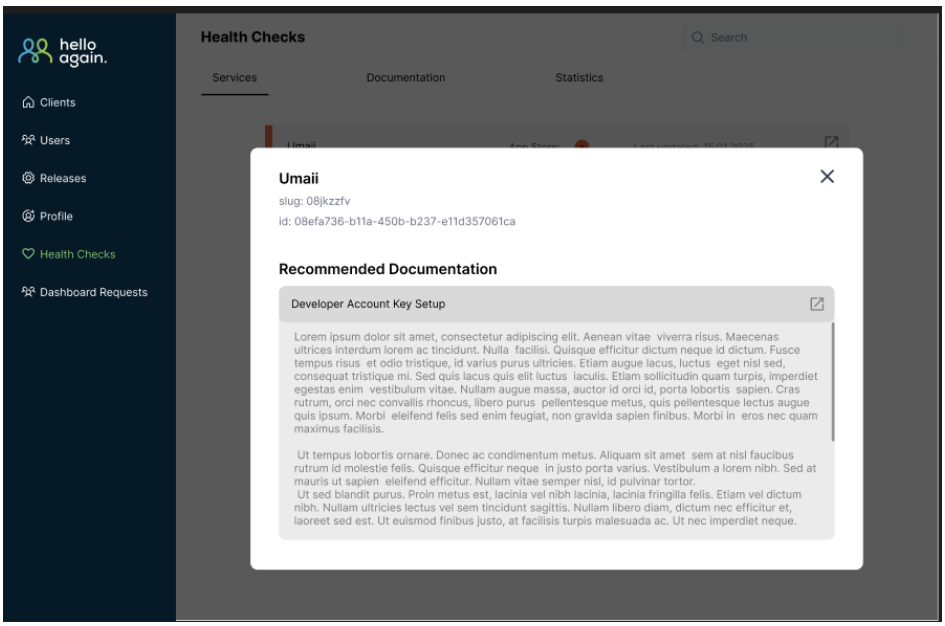
Um zu Dokumentationsseite zu gelangen kann der Tab gewechselt werden, allerdings werden hier alle vorhandenen Dokumentationen zur Fehlerbehebung angezeigt.





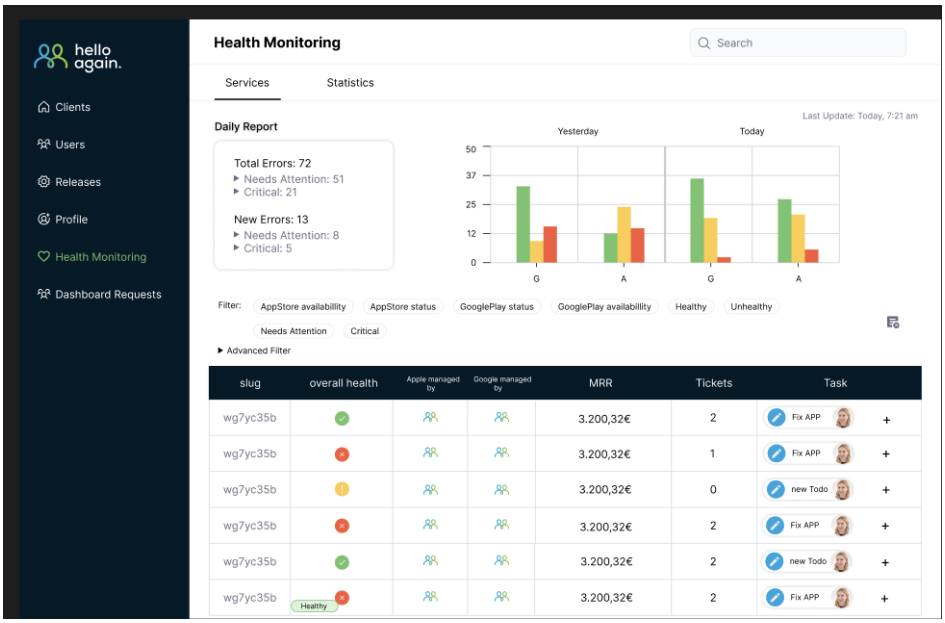
Als dritten Tab wurde eine Statistik Übersicht eingebaut bei der man die Downloads der letzten Tage sieht, sowie die Anzahl der App Ausfälle, damit man potenzielle Fehlerquellen zeitnahe erkennen und beseitigen kann.

Die Detailseite sieht sehr ähnlich aus zum ersten Workflow-Konzept allerdings mit dem Unterschied, dass man die Detailseite als Popup sieht. Unten rechts erscheint ein Button mit dem Namen Resolve Errors. Klickt man auf diesen, wird ein Fenster angezeigt, dass uns die Dokumentation zur Fehlerbehebung liefert.



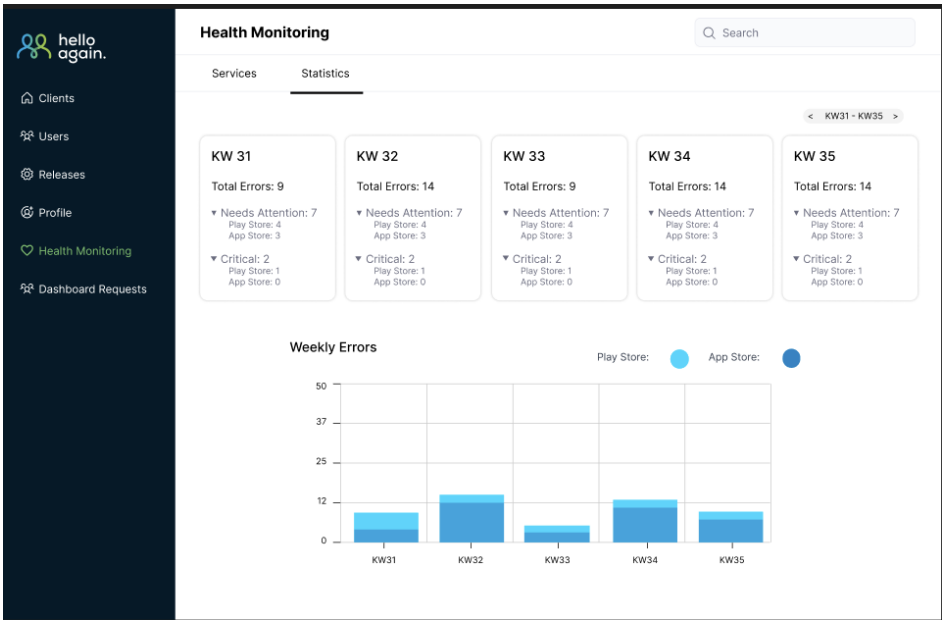
Iteration 2:

Zur Vorstellung beider Konzepte wurden wir in die Firma eingeladen und haben diese dort auch Vorgestellt. Es wurde schnell klar, dass wir eine Übersichtsseite sowie eine Detailseite brauchen würden mit einer erweiterten Kommentarfunktion und einem Link zur externen Fehlerbehebung, darum wurde sich auf eine Misch-Version der beiden Workflows geeinigt. Allerdings musste die Liste aller Kunden auf der Übersichtsseite noch etwas schmaler und informativer werden.



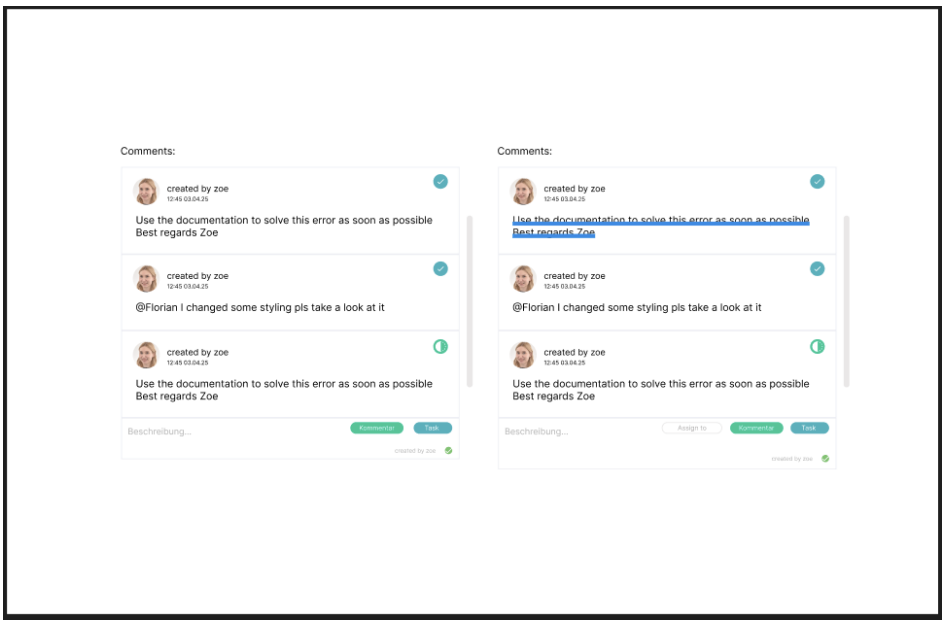
Hier sehen wir das verbesserte Dashboard-Design, dass nach dem ersten Feedback-Gespräch zustande kam. Es wurde eine Übersichtliche Statistik eingearbeitet sowie eine schmalere Liste mit mehr nützlichen Informationen für die Mitarbeiter. Dazu wurden

noch Quickfilter eingebaut, die ermöglichen sollen, dass man schnell alle fehlerhaften Apps herausfiltern kann, damit die Zeit, die die App nicht im Appstore verfügbar ist, so klein wie möglich gehalten wird, um so wenig potenziellen Umsatz wie möglich zu verlieren. Dazu gibt es noch die Advanced-Filter Option. Diese ist dafür da um nach konkreten Fehlern filtern zu können. Wenn man zum Beispiel weiß, dass das Liscenseagreement von Google geändert wurde, kann man mit den Advanced-Filtern danach suchen und diese Fehler beheben. Beim Overall-Health-Status wurde noch eine dritte Option eingführt neben Healthy und Healthy. Diese heißt Needs-Attention, dass heißt die App könnte bald aus dem Appstore oder Googleplaystore entfernt werden, da Lizenzen abgelaufen sind. Auch ein Schnellzugriff auf die TODOS soll gewährleistet sein.



Weiters gibt es einen eigenen Tab, der die Statistiken auf der Übersichtsseite nochmals erweitert und mit mehr Informationen darstellt.

Dazu wurde die Kommentar und TODO-Funktion nochmals überarbeitet in zwei verschiednen Version, hier wurde sich schlussendlich für diese Version (siehe Bild oben) entschieden.



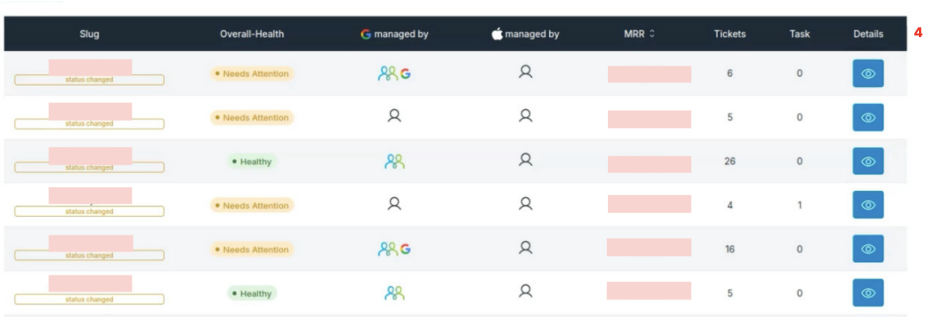
4.3 Dashboard

4.3.1 Frontend

Übersichtsseite

Tabellenansicht

Zu Beginn wurde die Tabelle erstellt auf der Übersichtsseite, hier werden die wichtigsten Daten der Kunden, für die Mitarbeiter dargestellt. Man muss beachten, dass das Design der finalen Version des Dashboards von der Figma Version abweicht, da die Firma intern Code guidelines hat, an die wir uns halten mussten. Ebenfalls gab es styling Guidelines und Komponenten die uns zu Verfügung gestellt wurden, wie zum Beispiel die Pagination.



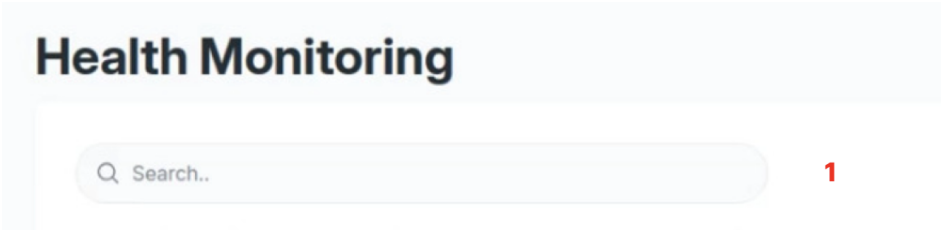
Hier werden angezeigt:

- der Slug (Eindeutiger Kundenbezeichner; zeigt auch ein status changed-Tag, falls sich der Status geändert hat)

- Overall-Health (Ampelstatus des Clients (Healthy, Needs Attention, Unhealthy, Critical))
- Google managed by (Zuständiges Team/Person für den Google-Account)
- Apple managed by (Zuständiges Team/Person für den Apple-Kanal)
- MRR (Monthly Recurring Revenue)
- Tickets (Anzahl offener Support-Tickets)
- Task (Anzahl offener Aufgaben)
- Details (Link zur Detailseite)

Da die Tabelle das Herzstück des Dashboards ist wurde sie als erstes implementiert und alles andere darauf aufgebaut. Bei der Tabelle wurde auch daraus geachtet, dass die Columns filterbar sind nach dem höchsten oder niedrigsten MRR.

Suchfunktion



Als nächstes wurde eine Suchfunktion implementiert, bei der man nach dem Slug filtern kann. Die Suchfunktion ist hierbei caseinsensitive.

Quickfilter

Um den Mitarbeitern eine möglichst schnelle Filteroption zu bieten, gibt es die Quickfilter, diese sind als multiselect implementiert worden. Das heißt man kann zum Beispiel nach allen Apps filtern, die unhealthy sind und mit status changed geflagged sind. Das bedeutet, die App war gestern noch auf Healthy oder Needs Attention und ist heute aus den Stores geflogen.

Was bedeuten die Quickfilter Optionen?

- Healthy (Die App weist keine Fehler auf)
- Needs Attention (Die App wird bald aus den Stores entfernt)

- Critical (Die App wurde aus den Stores entfernt und ist nichtmehr verfügbar)
- Unhealthy (Der Status der App ist entweder Needs Attention oder Critical. Hier muss etwas getan werden!)
- Google Availability Issues (Der Google Playstore weist Probleme bei der App auf)
- Apple Availability Issues (Der Appstore weist Probleme bei der App auf)
- Status Changed (Der Status hat sich zum Vortag verändert)

Das Filtern wird hier vom Frontend übernommen, da wir alle Daten der Tabelle benötigen und diese schon in das Frontend geladen haben. Würden wir wieder eine Request an das Backend schicken, würden wir unnötig Zeit des Benutzers verschwenden.

Advanced Filter

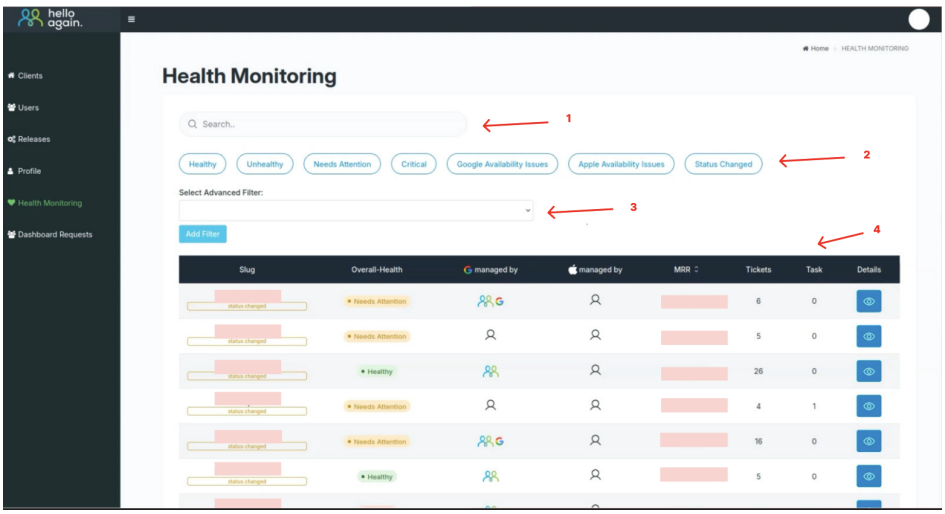
Die Advanced Filter wurden erst zum Schluss implementiert, da diese Filteroption durch das Backend passiert. Wir bekommen alle vorhandenen Fehler die vorkommen vom Backend ins Frontend und diese werden als zweiteilige Filteroption dargestellt. Zuerst wählt man aus nach was man Filtern möchte zum Beispiel Google Availability und danach muss man auswählen welchen spezifischen Error Code man ansehen möchte. Hier auch wieder ein Beispiel man hat Google Availability ausgewählt mit dem spezifischen Error Code License Agreement ran out. Das Frontend hat keinen Zugriff auf die Error Codes der einzelenen Apps, daher wird eine Anfrage an das Backend geschickt mit dem Error Code nachdem gefiltert werden soll. Anschließend wird die Liste neu gefiltert. Da wir eine sehr große menge an Daten durchgehen müssen, dauert dieser Vorgang auch dementsprechend lange.

Finale Version der Übersichtsseite

Detailseite

Status Changed

Am Anfang der Detailseite befindet sich die Status-Changed Flag. Dieser wird nochmal visuell hervorgehoben und zeigt an wie sich die App im gegensatz zum Vortag verändert



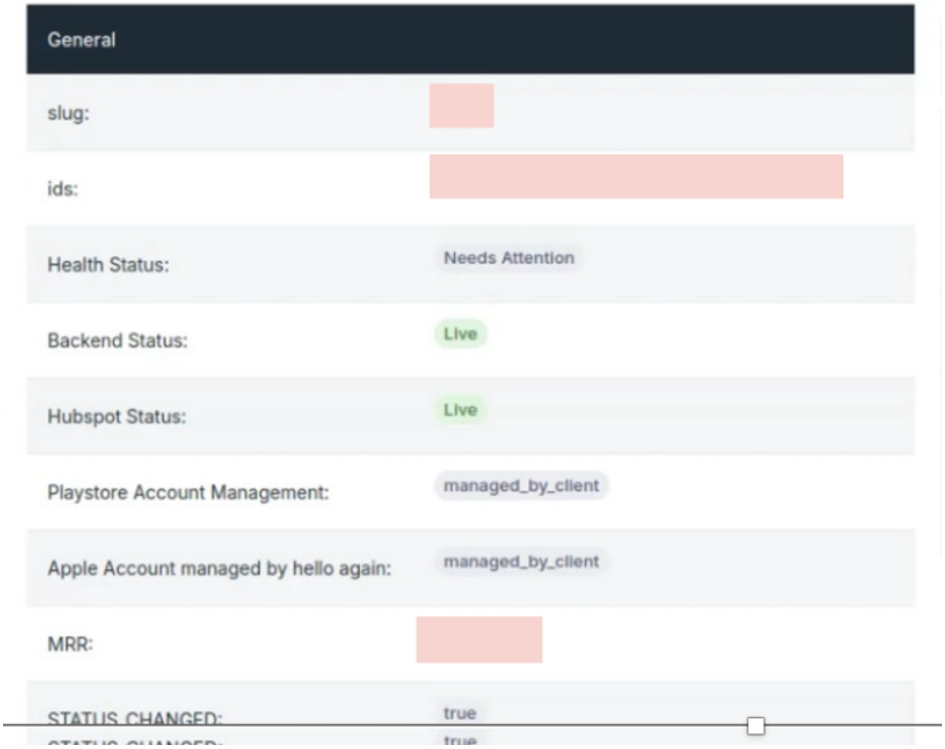
hat. Sollte die App den Status von Healthy zu Unhealthy verändern wird der Error Code, der für diesen Fehler verantwortlich ist auch angezeigt. Somit weiß der Benutzer sofort wo das Problem liegt. Status Changed wird allerdings auch angezeigt, wenn die App sich von Unhealthy zu Healthy ändert, so können die Mitarbeiter zum Beispiel nach der App, die gestern noch Fehlerhaft war suchen und sehen auf den ersten Blick, ob sich ihr Status verändert hat. Die Status changed Flags werden hier über eine Fetch-Request ins Frontend geladen.

General Data

Unter dem Status Changed Flag werden nochmals generelle Daten der App angezeigt, die auf der Übersichtsseite schon einsehbar sind. Hier wurde darauf geachtet, dass die Daten jederzeit erweiterbar sind. Das heißt, wenn wir aus dem Backend ergänzend noch YRR Daten bekommen würden, würde das Frontend sie ohne fremdeinwirkung eines Entwicklers darstellen.

Kommentar Sektion

Die Kommentarsektion ist eine der wichtigsten Funktionen der Detailsseite. Hier können Kommentare und Tasks erstellt werden, die ebenso in Asana erstellt werden. Ebenso werden die Kommentare und Tasks, die bereits in Asana vorhanden sind dargstellt. Wenn man einen Task erstellt kann man auch noch einen Assignee wählen, dem dieser Task dann zugeteilt wird. Hierzu bekommen wir aus dem Backend alle Namen der



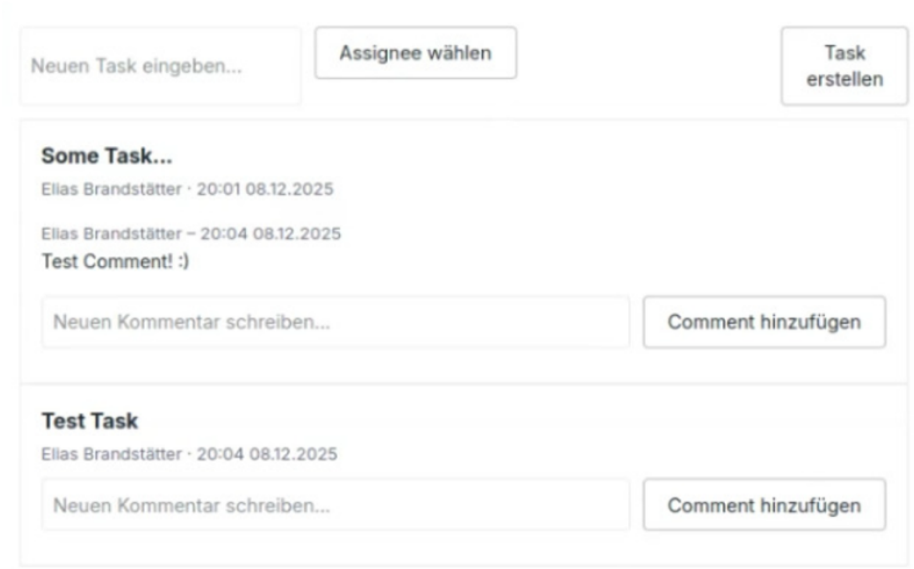
Mitarbeiter, die auf dieses Asana Board Zugriff haben. Wenn wir einen Task erstellen, wird der Text, der Slug und der Name des Assignees an das Backend übergeben, dass dan anschließend den Task in Asana erstellt.

Health-Checks

Eine weitere wichtige Funktion der Detailseite sind die Health-Checks. Hier wird der Status farblich gekennzeichnet, dass man sofort erkennt welche App Probleme wirft. Wenn ein Error vorliegt wird immer dessen Fehlermeldung angezeigt, sowie ein Link zu Fehlerbehebung. Drückt man auf den link wird man zur passenden Fehlerbeheungsdokumentation oder Anleitung in Confluence weitergeleitet. Diese Health-Checks sowie die passenden Links der Doku werden vom Backend bereitgestellt.

Tasks und Tickets

Es wurde außerdem gewünscht, dass die Tasks und Tickets dargestellt werden, da alle 3 miteinander intern verlinkt sind. Daher wird der Text beziehungsweise die Beschreibung der jeweils offenen Tickets angezeigt und die Links zur jeweiligen Plattform. Wichtig ist zu betonen, dass hier nur offenene Tickets dargestellt werden.



Finale Version der Detailsseite

Dies ist die finale Endversion der Detailseite.

4.4 Backend-Implementierung

Das Backend bildet die zentrale Verarbeitungsschicht des Health-Monitoring-Systems. Es ist dafür verantwortlich, Daten aus verschiedenen externen Quellen abzurufen, zusammenzuführen, zu verarbeiten und strukturiert an das Frontend bereitzustellen. Dabei fungiert es als einzige Schnittstelle zwischen der Datenhaltung in Google BigQuery, den externen Systemen wie Asana und HubSpot sowie dem Dashboard im Frontend.

4.4.1 Backend-Struktur

Das Backend ist als einzelne Python-Anwendung organisiert, deren Einstiegspunkt die Datei `main.py` bildet. Dort sind sämtliche API-Endpunkte, Hilfsfunktionen und die Konfiguration der Anwendung definiert. Ergänzend dazu existiert ein separates Konfigurationsmodul `config/confluence_links.py`, das die Zuordnung von Health-Check-Fehlercodes zu internen Confluence-Dokumentationsseiten verwaltet.

Tägliche Statusschnappschüsse werden im Verzeichnis `health_snapshots/` als CSV-Dateien abgelegt. Dieses Verzeichnis wird automatisch angelegt, sofern es noch nicht existiert, und dient als einfaches dateibasiertes Persistenzsystem für den Vergleich von Statusänderungen über Zeit.

Health Checks	Status
Playstore Availability Check:	OK
Appstore Availability Check:	OK
RSKV Safety Device Failed Check:	OK
Apple Store Status:	OK
Google Play Status:	THE_CALLER_DOES_NOT_HAVE_PERMISSION Link
Google Play Policy Status:	—
RSKV Invalid Check:	OK
Fetch Download Numbers Appstore:	OK
Fetch Download Numbers Playstore:	—

Tickets: 4			
	Hubspot	Asana	Jira
	Hubspot	Asana	Jira
	Hubspot	Asana	Jira
	Hubspot	Asana	Jira

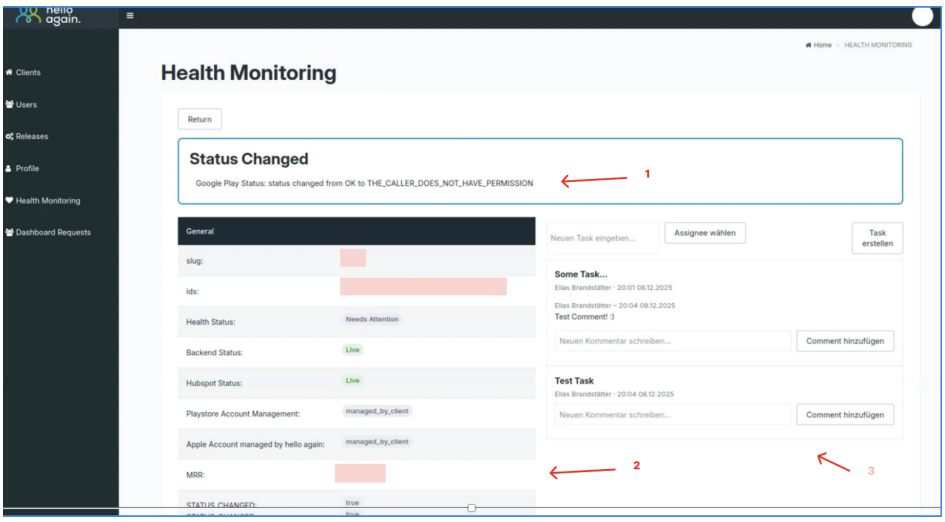
4.4.2 Konfiguration und Umgebungsvariablen

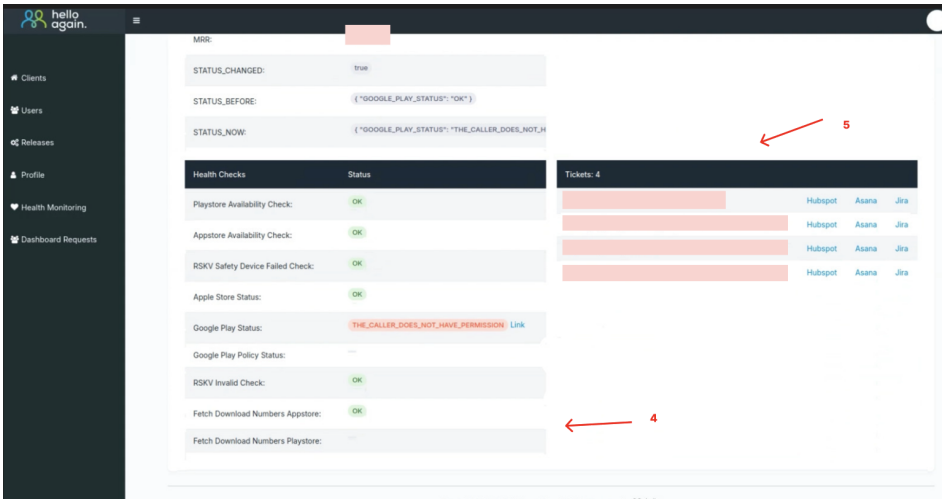
Alle systemkritischen Konfigurationswerte sind im Backend zentral verwaltet und nicht direkt im Quellcode hinterlegt. Die Verbindungsparameter für Google BigQuery - darunter die Projekt-ID `dwh-helloagain`, die Dataset-ID `health_monitoring` sowie der Standort der Daten - sind als Konstanten am Beginn der Anwendung definiert.

Zugangsdaten für externe Systeme werden über Umgebungsvariablen bereitgestellt. Der Asana Access Token wird beispielsweise über `os.getenv()` aus der Laufzeitumgebung gelesen:

```
ASANA_ACCESS_TOKEN = os.getenv('ASANA_ACCESS_TOKEN')
```

Asana-spezifische IDs wie die Workspace-ID, die Projekt-ID sowie die IDs der verwendeten Sections und Custom Fields sind ebenfalls als Konstanten hinterlegt. Aus





Entwicklungsgründen im Rahmen der Diplomarbeit wurde hier auf eine vollständige Auslagerung in Umgebungsvariablen verzichtet.

4.4.3 CORS-Konfiguration

Da Frontend und Backend in der Entwicklungsumgebung auf unterschiedlichen Ports betrieben werden, muss das Backend Cross-Origin Resource Sharing (CORS) explizit erlauben. CORS ist ein Sicherheitsmechanismus moderner Browser, der verhindert, dass Webseiten Anfragen an andere Domains oder Ports stellen, als jene, von der sie geladen wurden.

Im Backend wird CORS über die FastAPI-Middleware konfiguriert:

```
app.add_middleware(
CORSMiddleware,
allow_origins=["http://localhost:8080"],
allow_credentials=True,
```

```
allow_methods=["*"],
allow_headers=["*"],
)
```

In der aktuellen Konfiguration ist ausschließlich der lokale Entwicklungsserver des Frontends als erlaubter Ursprung eingetragen. Für den produktiven Betrieb wurde diese Konfiguration entsprechend auf die tatsächliche Domain des Frontends angepasst.

4.4.4 Allgemeine Architekturprinzipien

Die Architektur des Backends folgt einigen grundlegenden Prinzipien, die sich durch die gesamte Implementierung ziehen.

Klare Trennung der Verantwortlichkeiten

Jeder API-Endpunkt übernimmt eine klar definierte Aufgabe. Die Datenabfrage aus BigQuery, die Verarbeitung der Rohdaten und die Zusammenstellung der API-Antwort sind innerhalb eines Endpunkts logisch getrennt, auch wenn sie in derselben Funktion stattfinden. Wiederkehrende Operationen - etwa das Abrufen aller HubSpot-Tickets oder das Formatieren von Health-Check-Werten - sind in separate Hilfsfunktionen ausgelagert, die von mehreren Endpunkten gemeinsam genutzt werden können.

Backend als zentrale Integrationsschicht

Eine bewusste Entscheidung in der Architektur ist, dass das Frontend keinerlei direkte Verbindung zu externen Systemen aufbaut. Alle Daten - ob aus BigQuery, Asana oder HubSpot - werden ausschließlich über das Backend bezogen. Dies hat mehrere Vorteile: Erstens bleibt die gesamte Authentifizierungslogik im Backend zentralisiert, wo Zugangsdaten sicher verwaltet werden können. Zweitens kann das Backend die Daten aus verschiedenen Quellen zu einer einzigen strukturierten Antwort zusammenführen, sodass das Frontend nur einen einzigen API-Aufruf benötigt, um alle relevanten Informationen für eine App zu erhalten. Drittens können Änderungen an externen APIs oder Datenstrukturen im Backend abgefangen werden, ohne dass das Frontend angepasst werden muss.

Datenaggregation zur Laufzeit

Die Zusammenführung von Daten aus verschiedenen Quellen findet bewusst zur Laufzeit statt, also beim Eingang jeder API-Anfrage. Bei jedem Aufruf des `/health-checks` Endpunkts werden aktuelle Daten aus BigQuery geladen und die neuesten Ticket- und Task-Zahlen aus HubSpot und Asana werden hinzugefügt. Dieser Ansatz stellt sicher, dass das Dashboard stets den aktuellsten verfügbaren Zustand widerspiegelt, auch wenn sich Daten in den externen Systemen kurzfristig ändern.

Einheitliches Datenformat

Alle API-Antworten werden als JSON-Objekte zurückgegeben, wobei der Kunden-Slug als Primary-Key dient. Diese Struktur ermöglicht es dem Frontend, gezielt auf einzelne Kunden zuzugreifen und die Daten effizient weiterzuverarbeiten. Fehlende oder nicht verfügbare Werte werden als `null` zurückgegeben, anstatt das entsprechende Feld wegzulassen, was eine konsistente Verarbeitung im Frontend ermöglicht.

4.5 API-Endpunkte

Die REST-API des Health-Monitoring-Systems besteht aus mehreren Endpunkten, welche sich in drei funktionale Gruppen unterteilen lassen:

- Endpunkte für die Übersichtsansicht aller Kunden-Apps
- Endpunkte für die Detailansicht einzelner Kunden
- Endpunkte für die Asana-Integration

Alle Endpunkte geben ihre Ergebnisse als JSON zurück und sind über HTTP erreichbar.

Die Definition eines Endpunkts in FastAPI folgt einem einheitlichen Muster: Über einen Dekorator wird die HTTP-Methode und der Pfad festgelegt, die darunterliegende Python-Funktion übernimmt die Verarbeitungslogik.

Listing 1: Grundstruktur eines FastAPI-Endpunkts

```
1 @app.get("/beispiel/{parameter}")
2 def endpunkt_funktion(parameter: str):
3 # Verarbeitungslogik
4 return {"key": "value"}
```

4.5.1 GET /health-checks

Der `/health-checks`-Endpunkt ist der zentrale Endpunkt des Systems. Er wird beim Laden der Dashboard-Übersicht aufgerufen und liefert den aggregierten Status aller überwachten Kunden-Apps in einer einzigen Antwort.

Listing 2: Definition des /health-checks Endpunkts

```
1  @app.get("/health-checks")
2  def get_health_checks(
3  filters: Optional[List[str]] = Query(None)
4  ):
```

Datenabfrage

Beim Aufruf dieses Endpunkts wird zunächst eine SQL-Abfrage gegen die BigQuery-Tabelle `health_monitoring.health_data` ausgeführt, die für jede Kunden-App die wesentlichen Statusfelder liefert:

Listing 3: BigQuery-Abfrage im /health-checks Endpunkt

```
1  SELECT
2  slug,
3  PLAYSTORE_AVAILABILITY_CHECK,
4  APPSTORE_AVAILABILITY_CHECK,
5  RKSV_SAFETY_DEVICE_FAILED_CHECK,
6  APPLE_STORE_STATUS,
7  GOOGLE_PLAY_STATUS,
8  GOOGLE_PLAY_POLICY_STATUS,
9  RKSV_INVALID_CHECK,
10 FETCH_DOWNLOAD_NUMBERS_APPLE_APP_STORE,
11 FETCH_DOWNLOAD_NUMBERS_GOOGLE_PLAY_STORE,
12 Health_Status,
13 playstore_account,
14 apple_account,
15 MRR
16 FROM `dwh-helloagain.health_monitoring.health_data`
```

Parallel dazu werden über zwei Hilfsfunktionen die aktuellen HubSpot-Ticketanzahlen und offenen Asana-Tasks geladen. Diese drei Datenquellen werden anschließend pro Kunden-Slug zu einem einzigen Dictionary zusammengeführt, das als Grundlage für die API-Antwort dient.

Aggregation der Datenquellen

Die Zusammenführung der Daten aus BigQuery, HubSpot und Asana findet innerhalb einer Schleife statt, die alle zurückgelieferten Zeilen der BigQuery-Abfrage durchläuft. Für jede App wird dabei geprüft, ob in den HubSpot-Tickets oder Asana-Tasks ein Eintrag mit dem entsprechenden Slug existiert, und die jeweilige Anzahl wird aufsummiert:

Listing 4: Aggregation von HubSpot-Tickets pro Kunde

```
hubspot_ticket_count = 0
if hubspot_tickets:
  for index in hubspot_tickets:
  if index.get("slug_ticket") == slug:
    hubspot_ticket_count = index.get("ticket_count")
```

Bei den Asana-Tasks wird zusätzlich geprüft, ob der jeweilige Task dem Abschnitt *Tasks* innerhalb des Asana-Projekts zugeordnet ist, da das Projekt mehrere Abschnitte mit unterschiedlicher Bedeutung enthält:

Listing 5: Filterung von Asana-Tasks nach Section

```
if asana_tasks:
  for index in asana_tasks:
    task_slug = index.get("custom_fields", [{}])[1].get("text_value")
    if task_slug == slug:
    section_name = index.get("memberships", [{}])[0].get("section",
        {}).get("name", "")
    if section_name.lower() == "tasks":
        asana_tasks_count += 1
```

Filterlogik

Der Endpunkt unterstützt optionale Filterparameter, die als Query-Parameter in der URL übergeben werden. Filter werden im Format `key:value` angegeben, wobei `key` dem Namen eines Statusfeldes entspricht und `value` dem gewünschten Wert. Es können mehrere Filter gleichzeitig übergeben werden; ein Kunden-Eintrag erscheint in der Antwort nur dann, wenn er alle angegebenen Filter erfüllt:

Listing 6: Filterlogik im /health-checks Endpunkt

```
if filters:
  filtered_results = {}
  for slug, data in results.items():
    match_all = True
    for f in filters:
      if ":" in f:
        key, value = f.split(":", 1)
        if str(data.get(key)) != value:
          match_all = False
          break
      else:
        if not data.get(f):
          match_all = False
          break
    if match_all:
      filtered_results[slug] = data
  results = filtered_results
```

4.5.2 GET /filter_data

Damit das Frontend dem Nutzer ausschließlich tatsächlich vorhandene Filterwerte anbietet, stellt das Backend den Endpunkt `/filter_data` bereit. Dieser liest alle

aktuellen Einträge aus BigQuery und extrahiert daraus für jedes Health-Check-Feld die Menge aller vorkommenden Werte:

Listing 7: Extraktion verfügbarer Fehlercodes

```
error_codes = {
  "PLAYSTORE_AVAILABILITY_CHECK": set(),
  "APPSTORE_AVAILABILITY_CHECK": set(),
  "RKSV_SAFETY_DEVICE_FAILED_CHECK": set(),
  "APPLE_STORE_STATUS": set(),
  "GOOGLE_PLAY_STATUS": set(),
  "GOOGLE_PLAY_POLICY_STATUS": set(),
  "RKSV_INVALID_CHECK": set(),
  "FETCH_DOWNLOAD_NUMBERS_APPLE_APP_STORE": set(),
  "FETCH_DOWNLOAD_NUMBERS_GOOGLE_PLAY_STORE": set(),
}

for row in rows:
  for key in error_codes.keys():
    value = getattr(row, key, None)
    if value:
      error_codes[key].add(str(value))
```

Das Ergebnis ist ein strukturiertes JSON-Objekt, das pro Health-Check-Kategorie eine Liste aller vorkommenden Fehlercodes enthält. Leere Kategorien - also Felder, bei denen alle Apps den Status `OK` aufweisen - werden aus der Antwort herausgefiltert, da sie für die Filterauswahl im Frontend nicht relevant sind. Dadurch wird sichergestellt, dass die Filteroptionen im Dashboard stets den realen Datenzustand widerspiegeln und keine leeren oder ungültigen Filterkombinationen angeboten werden.

4.5.3 GET /client_details/{slug}

Der `/client_details`-Endpunkt liefert die vollständige Detailansicht einer einzelnen Kunden-App. Im Gegensatz zum `/health-checks`-Endpunkt, der eine kompakte Übersicht aller Apps liefert, enthält die Detailansicht alle verfügbaren Informationen zu einem bestimmten Kunden - darunter aufbereitete Health-Check-Ergebnisse mit Confluence-Links, verknüpfte Ticket-Informationen sowie eine Gegenüberstellung des aktuellen Status mit dem zuletzt gespeicherten Snapshot.

Parametrisierte Datenbankabfrage

Der Slug des gewünschten Kunden wird als Pfadparameter übergeben und direkt in die BigQuery-Abfrage eingebettet. Um SQL-Injection zu verhindern, erfolgt dies nicht durch einfache String-Interpolation, sondern über parametrisierte Abfragen mit dem `QueryJobConfig`-Mechanismus der BigQuery-Clientbibliothek:

Listing 8: Parametrisierte BigQuery-Abfrage im /client_details Endpunkt

```
query = """
        SELECT *
        FROM `dwh-helloagain.health_monitoring.health_data`
        WHERE slug = @slug \
        """
job_config = bigquery.QueryJobConfig(
  query_parameters=[bigquery.ScalarQueryParameter("slug", "STRING", slug)]
)
rows = list(client.query(query, job_config=job_config).result())
```

Wird kein Eintrag für den angegebenen Slug gefunden, gibt der Endpunkt einen HTTP-404-Fehler zurück.

Aufbereitung der Health-Check-Ergebnisse

Die Hilfsfunktion `format_health_check()` prüft, ob ein Statuswert von `OK` abweicht, und fügt in diesem Fall einen direkten Link zu einem internen Confluence-Artikel hinzu, der den entsprechenden Fehlerbehebungsprozess beschreibt:

Listing 9: Hilfsfunktion format_health_check

```
def format_health_check(value: str, link: Optional[str] = None):
  if value and value != "OK":
    return [value, link or ""]
  return value
```

Ist der Status `OK`, wird der Wert unverändert zurückgegeben. Bei einem Fehler hingegen gibt die Funktion eine Liste zurück, die sowohl den Fehlerwert als auch den zugehörigen Confluence-Link enthält. Das Frontend kann anhand dieser Struktur erkennen, ob und welche weiterführende Dokumentation verfügbar ist, und den Link direkt im Dashboard darstellen.

Statusvergleich und Change Detection

Neben den aktuellen Statuswerten enthält die Antwort dieses Endpunkts auch Informationen darüber, ob und wie sich der Status einer App seit dem letzten Snapshot verändert hat. Die dafür notwendige Logik ist Teil des Snapshot-Systems, das in Kapitel 4.6 ausführlich beschrieben wird. Das Ergebnis des Vergleichs wird in drei Feldern in der API-Antwort abgebildet:

Listing 10: Statusänderungsfelder in der API-Antwort

```
result = {
  ...
  "STATUS_CHANGED": status_changed,
  "STATUS_BEFORE": status_before,
  "STATUS_NOW": status_now,
...
}
```

Das Feld `STATUS_CHANGED` ist ein boolescher Wert, der angibt, ob überhaupt eine Änderung vorliegt. `STATUS_BEFORE` und `STATUS_NOW` sind Dictionaries, die jeweils nur die tatsächlich veränderten Felder enthalten - mit dem Wert vor und nach der Änderung. Das Frontend kann dadurch gezielt nur jene Felder hervorheben, die sich tatsächlich verändert haben.

4.5.4 GET /client_tickets/{slug}

Dieser Endpunkt liefert alle offenen HubSpot-Tickets, die einem bestimmten Kunden zugeordnet sind. Er wird sowohl intern vom `/client_details`-Endpunkt aufgerufen als auch direkt vom Frontend genutzt, um die Ticket-Liste in der Detailansicht darzustellen.

Listing 11: BigQuery-Abfrage für Kunden-Tickets

```
1  SELECT *
2  FROM `dwh-helloagain.health_monitoring.hubspot_tickets`
3  WHERE slug_ticket = @slug
4  AND hs_pipeline_stage IN ('4', '1071985524')
```

Die Filterung nach Pipeline-Stages stellt sicher, dass ausschließlich aktive Supportfälle zurückgegeben werden. Abgeschlossene oder archivierte Tickets werden nicht berücksichtigt, da sie für den aktuellen App-Status nicht relevant sind. Die zurückgelieferten Tickets enthalten neben dem Ticket-Namen auch direkte Links zu HubSpot, Jira und Asana, sofern diese im jeweiligen Ticket hinterlegt sind:

Listing 12: Aufbereitung der Ticket-Daten

```
1  tickets = [
2    {
3      "name": row.subject,
4      "jira": row.jira_link,
5      "asana": row.asana_link,
6      "hubspot": (
7        f"https://app.hubspot.com/contacts/XXXXXXX/record/0-5/{row.id}"
8        if row.id else None
9      ),
10   }
11   for row in rows
12 ]
```

Der HubSpot-Link wird dabei dynamisch aus der Ticket-ID zusammengesetzt, da HubSpot keine direkte Link-Spalte in der Datenbank speichert. (Teil des Links wurde zensiert)

4.5.5 Asana-Endpunkte

Die Asana-Integration umfasst drei Endpunkte, die gemeinsam das Aufgabenmanagement im Dashboard abdecken. Sie ermöglichen das Abrufen von Tasks und Kommentaren, das Erstellen neuer Einträge sowie das Laden der verfügbaren Mitarbeiterliste.

GET /asana/tasks/{slug}

Dieser Endpunkt liefert alle Asana-Tasks, die einem bestimmten Kunden-Slug zugeordnet sind, inklusive der zugehörigen Kommentare. Da die Asana API keine direkte Filterung nach Custom Fields unterstützt [?], wird die gesamte Taskliste des konfigurierten Projekts geladen und anschließend im Backend gefiltert:

Listing 13: Filterung von Asana-Tasks nach Slug

```
for task in tasks:
  # Match slug via custom field
  task_slug = next(
    (cf.get("text_value") for cf in task.get("custom_fields", [])
     if cf.get("name", "").lower() == "slug"),
    None
  )
  if task_slug != slug:
    continue
```

Zu jedem passenden Task werden anschließend die zugehörigen Kommentare über die Asana *Stories*-API geladen [?]. Da Stories in Asana nicht nur Nutzerkommentare, sondern auch automatisch generierte Systemeinträge wie Statusänderungen oder Zuweisungshinweise umfassen, werden diese anhand des Feldes `resource_subtype` herausgefiltert:

Listing 14: Filterung von Nutzerkommentaren aus Asana Stories

```
for story in stories:
  subtype = story.get("resource_subtype") or story.get("type")

  # Only include real user-written comments
  if subtype not in ("comment", "comment_added"):
    continue

  task_entry["Comments"].append({
  "created_by": story.get("created_by", {}).get("name", "Unknown"),
  "text": story.get("text", ""),
  "type": "comment",
  "date": format_asana_date(story.get("created_at"))
  })
```

Datumsangaben werden über die Hilfsfunktion `format_asana_date()` aus dem ISO-8601-Format von Asana in ein für das Dashboard lesbares Format umgewandelt.

POST /asana/create_comment_or_task

Dieser Endpunkt ermöglicht das Erstellen neuer Asana-Einträge direkt aus dem Dashboard heraus. Über den Parameter `type` im Request-Body wird unterschieden, ob ein neuer Task oder ein Kommentar zu einem bestehenden Task erstellt werden soll.

Im Fall eines neuen Tasks werden die notwendigen Metadaten - darunter das Asana-Projekt, der Abschnitt sowie der Kunden-Slug als Custom Field - automatisch vom Backend gesetzt:

Listing 15: Erstellung eines neuen Asana-Tasks

```
task_data = {
  "name": text,
  "projects": [ASANA_PROJECT_ID],
  "custom_fields": {
    ASANA_SLUG_CUSTOM_FIELD_ID: slug,
    ASANA_CREATED_BY_CUSTOM_FIELD_ID: user_gid
  },
  "memberships": [{
    "project": ASANA_PROJECT_ID,
    "section": ASANA_TASKS_SECTION_ID
  }]
}
```

Das automatische Setzen des Slugs als Custom Field ist dabei besonders wichtig, da dieser Wert die Grundlage für die spätere Zuordnung des Tasks zu einer Kunden-App bildet. Würde dieses Feld fehlen, wäre der Task in der Dashboard-Ansicht nicht mehr auffindbar.

Bei einem Kommentar wird stattdessen die Asana *Stories*-API verwendet, um den Text direkt an einen bestehenden Task zu knüpfen. Die `task_gid` des Ziel-Tasks muss dabei im Request-Body mitgeliefert werden.

In beiden Fällen wird der aktuell authentifizierte Nutzer automatisch als Ersteller gesetzt. Dazu wird zu Beginn jeder Schreiboperation der `/users/me` Endpunkt der Asana API aufgerufen, um die `gid` und den Namen des eingeloggten Nutzers zu ermitteln:

Listing 16: Ermittlung des aktuellen Nutzers über die Asana API

```
me = client.users.me()
user_gid = me["gid"]
user_name = me["name"]
```

GET /asana/employees

Dieser Endpunkt liefert eine Liste aller Mitglieder des konfigurierten Asana-Workspace, jeweils mit ihrer `gid` und ihrem Namen. Die Liste wird im Frontend verwendet, um

beim Erstellen eines neuen Tasks eine Zuweisung an einen konkreten Mitarbeiter zu ermöglichen. Da die Mitarbeiterliste direkt von der Asana API bezogen wird, spiegelt sie stets den aktuellen Stand des Workspaces wider - neue Mitarbeiter erscheinen automatisch in der Auswahl, ohne dass Änderungen am System notwendig sind.

4.6 Datenmanagement

Das Datenmanagement des Health-Monitoring-Systems gliedert sich in zwei Bereiche: die Abfrage und Verarbeitung aktueller Daten aus Google BigQuery sowie ein dateibasiertes Snapshot-System, das den Vergleich von Statusänderungen über Zeit ermöglicht.

4.6.1 Google BigQuery

Sämtliche Monitoring-Daten des Systems werden in Google BigQuery gespeichert und von dort über SQL-Abfragen abgerufen. Das Backend greift dabei auf drei Tabellen im Dataset `health_monitoring` zurück:

- `health_data`: enthält die aggregierten Health-Check-Ergebnisse aller Kunden-Apps, darunter plattformspezifische Statuswerte, den konsolidierten `Health_Status` sowie Account- und MRR-Informationen
- `hubspot_tickets`: enthält die aus HubSpot synchronisierten Support-Tickets mit Zuordnung zum jeweiligen Kunden-Slug
- weitere Konfigurationstabellen, die App- und Account-Metadaten bereitstellen

Der Zugriff auf BigQuery erfolgt über die offizielle Python-Clientbibliothek. Eine Verbindung wird dabei nicht dauerhaft offen gehalten, sondern bei jedem API-Aufruf neu aufgebaut:

Listing 17: Initialisierung des BigQuery-Clients

```
1  client = bigquery.Client(project=PROJECT_ID, location=LOCATION)
```

Die Angabe des geografischen Standorts über den Parameter `location` ist dabei notwendig, da die verwendeten BigQuery-Datasets in der europäischen Region gespeichert sind. Würde der Standort nicht explizit angegeben, könnten Abfragen fehlschlagen oder auf falsche Datenregionen zugreifen.

Datenverarbeitung nach dem Abruf

Die von BigQuery zurückgelieferten Rohdaten werden im Backend nicht direkt an das Frontend weitergegeben, sondern zunächst verarbeitet und in eine einheitliche Struktur gebracht. Das umfasst unter anderem die Auflösung von Array-Feldern, bei denen BigQuery mehrere Werte als Liste zurückgibt und das Backend den jeweils relevanten Wert extrahiert:

Listing 18: Beispiel von Extraktion von Array-Feldern aus BigQuery-Ergebnissen

```
1 "playstore_account_managements": (
2 playstore_account[0] if playstore_account else None
3 ),
4 "apple_dev_acct_managed_by_hello_again": str(
5 apple_account[0] == "managed_by_hello_again"
6 if apple_account else False
7 ),
```

Fehlende oder leere Werte werden als `None` gesetzt, um eine konsistente Verarbeitung im Frontend zu gewährleisten. Für den Zugriff auf Felder, die je nach Abfragekontext entweder als Attribut oder als Dictionary-Eintrag vorliegen können, wird eine eigene Hilfsfunktion `safe_get()` verwendet:

Listing 19: Hilfsfunktion safe_get() für flexiblen Feldzugriff

```
1 def safe_get(obj, key):
2 if hasattr(obj, key):
3 return getattr(obj, key)
4 if isinstance(obj, dict):
5 return obj.get(key)
6 return None
```

Diese Funktion ist notwendig, weil BigQuery-Ergebniszeilen je nach Abfrageart unterschiedliche Python-Objekte zurückliefern können - einmal als Zeileobjekt mit Attributzugriff, ein anderes Mal als Dictionary. Die Hilfsfunktion abstrahiert diesen Unterschied und verhindert Laufzeitfehler durch unerwartet fehlende Attribute.

4.6.2 Snapshot-System

Eine zentrale Anforderung des Health-Monitoring-Systems ist die Erkennung von Statusänderungen: Das Dashboard soll nicht nur den aktuellen Zustand einer App anzeigen, sondern auch darauf hinweisen, wenn sich dieser Zustand seit dem letzten bekannten Stand verändert hat. Um diese Funktionalität im Rahmen der Diplomarbeit zu realisieren, ohne eine eigene Historiendatenbank betreiben zu müssen, setzt das Backend auf ein einfaches dateibasiertes Snapshot-System.

Funktionsweise

Bei jedem Aufruf der relevanten API-Endpunkte wird der aktuelle Zustand der abgefragten Daten als CSV-Datei im Verzeichnis `health_snapshots/` gespeichert. Der Dateiname enthält dabei das aktuelle Datum im Format `YYYY-MM-DD`, sodass pro Tag genau eine Snapshot-Datei entsteht:

Listing 20: Dateinamen der Snapshot-Dateien

```
today_str = date.today().strftime("%Y-%m-%d")
today_file = os.path.join(
  CSV_DIR, f"health_checks_{today_str}.csv"
)
```

Existiert für den aktuellen Tag bereits eine Snapshot-Datei, wird keine neue erstellt - der erste Aufruf des Tages legt den Snapshot fest, alle weiteren Aufrufe lesen ihn lediglich. Dadurch wird sichergestellt, dass der Vergleich stets gegen den Zustand zu Beginn des Tages erfolgt und nicht gegen einen im Laufe des Tages aktualisierten Wert.

Es gibt zwei separate Snapshot-Typen: `health_checks_YYYY-MM-DD.csv` für die Übersichtsansicht aller Apps und `client_details_YYYY-MM-DD.csv` für die Detailansicht einzelner Kunden. Diese Trennung ist notwendig, da die Detailansicht deutlich mehr Felder enthält als die Übersicht und ein gemeinsamer Snapshot zu einer unnötig großen Datei führen würde.

Umgang mit fehlenden Snapshots

Da der Snapshot erst beim ersten API-Aufruf des Tages erstellt wird, kann es vorkommen, dass beim Vergleich kein Snapshot des Vortages existiert - etwa wenn das System an einem bestimmten Tag nicht aufgerufen wurde. In diesem Fall erstellt das Backend automatisch eine leere Snapshot-Datei mit den korrekten Spaltenköpfen, um Fehler beim späteren Einlesen zu vermeiden:

Listing 21: Automatische Erstellung eines leeren Snapshots

```
if not os.path.exists(yesterday_file):

  # Generate correct fieldnames based on today's data
  if new_data:
    fieldnames = ["slug"] + list(next(iter(new_data.values())).keys())
  else:
    fieldnames = ["slug"]

  # Create empty yesterday CSV file
  with open(yesterday_file, mode="w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()     # write CSV header only (empty snapshot)
```

Eine leere Snapshot-Datei führt beim Vergleich dazu, dass alle aktuellen Werte als neu gewertet werden, da kein Vortagswert zum Vergleich existiert. Das ist ein bewusst in Kauf genommenes Verhalten, das im Normalbetrieb - bei täglichem Systemaufruf zu einer festgelegten Uhrzeit - nicht auftreten wird.

Lesen und Schreiben von Snapshots

Das Lesen bestehender Snapshots erfolgt über Pythons `csv.DictReader`, der jede Zeile als Dictionary mit den Spaltenköpfen als Schlüssel einliest. Das Ergebnis wird in einem Dictionary gespeichert, das den Slug als Schlüssel und die zugehörigen Feldwerte als Werte enthält:

Listing 22: Einlesen eines bestehenden Snapshots

```
1  try:
2    with open(yesterday_file, mode="r", newline="") as f:
3      reader = csv.DictReader(f)
4      for row in reader:
5        slug = row.get("slug")
6        if not slug:
7          continue
8
9        old_data[slug] = {k: row[k] for k in row if k != "slug"}
```

Das Schreiben eines neuen Snapshots erfolgt analog über `csv.DictWriter`. Die Spaltenköpfe werden dabei dynamisch aus den Schlüsseln des aktuellen Datendictionaries abgeleitet, sodass eine Erweiterung der Datenstruktur nicht zu Anpassungen am Snapshot-Code führt.

Detailansicht: Aktualisierung des Snapshots

Bei der Detailansicht eines einzelnen Kunden verhält sich das Snapshot-System etwas anders als bei der Übersicht. Da die Detailansicht nur für einen bestimmten Kunden aufgerufen wird, enthält die zugehörige Snapshot-Datei nur die Einträge jener Kunden, deren Detailseite tatsächlich aufgerufen wurde. Beim Aufruf wird der Snapshot daher nicht neu erstellt, sondern aktualisiert: Existiert bereits ein Eintrag für den jeweiligen Slug, wird er überschrieben; andernfalls wird ein neuer Eintrag hinzugefügt:

Listing 23: Aktualisierung des Detailansicht-Snapshots

```
1  updated = False
2  for r in today_rows:
3    if r["slug"] == slug:
4      for k, v in new_data.items():
5        r[k] = v
6      updated = True
7      break
8  if not updated:
```

```
    today_rows.append(new_data)
```

4.6.3 Change Detection

Change Detection in der Übersichtsansicht

In der Übersichtsansicht wird pro App lediglich ein boolescher Wert `STATUS_CHANGED` berechnet, der angibt, ob sich irgendeines der relevanten Felder seit dem letzten Snapshot verändert hat:

Listing 24: Berechnung von STATUS_CHANGED in der Übersicht

```
status_changed = any(
  str(old_val.get(field)) != str(new_field_value)
  for field, new_field_value in new_val.items()
)

new_val["STATUS_CHANGED"] = status_changed
```

Dieser Wert reicht für die Übersicht aus, da das Dashboard dort lediglich eine visuelle Markierung setzt, ohne die konkreten Änderungen im Detail darzustellen.

Change Detection in der Detailansicht

In der Detailansicht geht das System einen Schritt weiter. Hier werden nicht nur geänderte Felder erkannt, sondern auch die konkreten Werte vor und nach der Änderung gespeichert. Dabei werden bewusst nur jene Felder in den Vergleich einbezogen, die aus BigQuery stammen. Dynamisch berechnete Werte wie `open_tasks` und `open_hubspot_tickets` werden vom Vergleich ausgeschlossen, da sich diese Werte unabhängig vom eigentlichen App-Status ändern können und andernfalls zu falschen Statusänderungsmeldungen führen würden:

Listing 25: Feldweiser Vergleich in der Detailansicht

```
for field in base_fields:
  if field == "slug":
    continue
  old_val = normalize(old_data.get(field))
  new_val = normalize(new_data.get(field))
  if old_val != new_val:
    status_before[field] = old_data.get(field)
    status_now[field] = new_data.get(field)

status_changed = len(status_before) > 0
```

Das Ergebnis sind die zwei Dictionaries `STATUS_BEFORE` und `STATUS_NOW`, die jeweils nur die tatsächlich veränderten Felder enthalten. Das Frontend kann diese Informationen

direkt nutzen, um dem Nutzer eine übersichtliche Gegenüberstellung der Änderungen anzuzeigen, ohne selbst eine Vergleichslogik implementieren zu müssen.

4.7 Sicherheit und Zugriff

4.7.1 Einleitung

Im Rahmen unserer Diplomarbeit bei Hello Again wurde dem Thema Sicherheit und Datenschutz von Beginn an ein hoher Stellenwert gegeben. Bereits zu Beginn der Diplomarbeit fand eine Einschulung statt, in der uns die Richtlinien zum sicheren Umgang mit Daten, Systemen und Tools erklärt wurden. Diese Richtlinien betrafen sowohl den Umgang mit sensiblen Kundendaten als auch die erlaubten Tools.

4.7.2 Datenschutz und der Umgang mit Kundendaten

Einer der wichtigsten Punkte betraf den Umgang mit Kundendaten. Uns wurde explizit erklärt, dass Kundendaten zu keinem Zeitpunkt in externe KI-Systeme eingegeben werden dürfen. Konkret wurde dabei ChatGPT als Beispiel genannt; ein öffentliches KI-Tool, das von OpenAI betrieben wird. Kundendaten unterliegen in der Europäischen Union der Datenschutz-Grundverordnung (DSGVO), die strenge Anforderungen an die Verarbeitung, Speicherung und Weitergabe personenbezogener Daten stellt. Werden diese Daten an Dienste weitergegeben, ohne dass eine entsprechende rechtliche Grundlage vorliegt, ist dies ein Datenschutzverstoß, der für das Unternehmen erhebliche rechtliche und finanzielle Konsequenzen haben kann. Das Vertrauen der Kunden in die Plattform von Hello Again sehr wichtig; jede Form von Databreach würde das Vertrauen beschädigen zwischen Kunde und Firma. Für uns heißt das, dass wir darauf achten mussten, bei der Nutzung von KI-Tools ausschließlich selbst erstellte Beispieldaten zu verwenden. Keine echten Kundendaten!

4.7.3 Erlaubte KI-Tools: Firmeninterne Lösungen

Hello Again setzt auf firmenintern verwaltete KI-Lösungen. Im Rahmen der Diplomarbeit durften waren zwei Tools zur Nutzung freigegeben. Google Gemini (bezahlte Firmenversion): Hello Again verwendet die kostenpflichtige, unternehmensseitig verwal-

tete Version von Google Gemini. Der Unterschied zur kostenlosen öffentlichen Version ist, dass Google mit Enterprise-Verträgen garantiert, dass die Daten nicht für das Training von Modellen zu verwendet werden. Claude als Konsolentool (bezahlte Version im Firmen-Environment): Zusätzlich duften wir Claude von Anthropic verwenden, jedoch ausschließlich in der bezahlten Version. Claude wurde dabei als Konsolentool eingesetzt, also für Debugging-Unterstützung.

Hier kann man erkennen, dass die Unternehmen die neuartigen KI-Tools nicht ablehnen sondern ihre Mitarbeiter ermutigen diese sinnvoll einzusetzen.

4.7.4 Sensibilisierung für Phishing-Angriffe

Ein weiterer Bestandteil der Einschulung war die Aufklärung über Phishing-Angriffe. Phishing bezeichnet den Versuch, Mitarbeiter durch gefälschte E-Mails, Links oder Webseiten dazu zu bringen, sensible Informationen herauszugeben zum Beispiel Passwörter, Zugangsdaten uvm. Für Unternehmen sind Phishing-Angriffe besonders gefährlich, weil sie häufig gezielt auf bestimmte Organisationen abgezielt sind. Wir wurden gebeten, bei jeder E-Mail mit unbekanntem Absender oder ungewöhnlichem Inhalt besonders Vorsicht zu sein, keine Links einfach so zu öffnen und Rücksprache mit unserem Betreuer zu halten.

4.7.5 Branches

Wir arbeiteten nicht direkt auf dem Main Branch des Projekts. Der Main Branch enthält den Code der das Dashboard am laufen hält. Direkte Änderungen an diesem Branch würden ein Risiko darstellen, da fehlerhafte Commits Auswirkungen auf das laufende Dashboard haben könnten bzw haben. Stattdessen wurde uns ein Branch namens health-monitoring bereitgestellt. Dieser Branch war unser main Arbeitsbereich. Er war vom Main Branch isoliert, sodass unsere Änderungen restliche Projekt nicht beeinflussen konnten. Innerhalb unseres health-monitoring-Branches legten wir uns zwei weitere Branches an, um eine klare Trennung zwischen Frontend und Backend zu gewährleisten, einen frontend-Branch für alle Arbeiten an UI und UX, sowie einen backend-Branch für die serverseitige Logik, API-Anbindungen und Datenbankoperationen. So konnten effizient Merge-Konflikte reduziert werden. Am Ende der Diplomarbeit wurde unser erarbeiteter Code nach der Überprüfung, durch die Devs in das offizielle Projekt gemerged.

4.7.6 Kein separates Mockup-Projekt

In vielen Firmen ist es üblich, Praktikanten oder Schülerinnen und Schülern ein Demoprojekt zur Verfügung zu stellen, an dem sie arbeiten können. In unserem Fall war dies firmenintern nicht machbar. Der Grund war personeller und zeitlicher Aufwand. Das Aufsetzen eines Mockup-Projekts hätte viel Zeit und Entwickler gekostet. Da diese Devs nicht verfügbar waren, entschied man sich für einen eigenen Branch. Diese Entscheidung hatte sowohl Vorteile als auch Nachteile. Der offensichtliche Vorteil liegt darin, dass wir mit echten firmeninternen Strukturen gearbeitet haben aber der Nachteil besteht darin, dass die Fehlertoleranz geringer ist. Schwerwiegende Fehler im Branch hätten den Merge-Prozess erschwert. Da aber die Entwickler unsere Commits regelmäßig begutachtet haben, hatten wir diese Probleme nicht.

5 Zusammenfassung

Aufzählungen:

- Itemize Level 1
  - Itemize Level 2
    - Itemize Level 3 (vermeiden)

1. Enumerate Level 1
   a. Enumerate Level 2
      i. Enumerate Level 3 (vermeiden)

Desc Level 1

Desc Level 2 (vermeiden)

Desc Level 3 (vermeiden)

Literaturverzeichnis

[1]

[2] Apple Inc. App store connect api, 2024. letzter Zugriff am 01.04.2026.

[3] Asana Inc. Asana developer documentation, 2024. letzter Zugriff am 31.03.2026.

[4] Asana Inc. Filtering and sorting custom fields, 2024. letzter Zugriff am 04.04.2026.

[5] Asana Inc. Stories, 2024. letzter Zugriff am 04.04.2026.

[6] Donald Chamberlin and Raymond Boyce. Sequel: A structured english query language. *Proceedings of ACM SIGFIDET*, 1974. letzter Zugriff am 31.03.2026.

[7] Samuel Colvin. Data validation and settings management using python type annotations. *Pydantic Documentation*, 2024. letzter Zugriff am 14.03.2026.

[8] Django Software Foundation. Asgi specification, 2023. letzter Zugriff am 14.03.2026.

[9] Encode OSS. Uvicorn documentation, 2024. letzter Zugriff am 14.03.2026.

[10] Evan You. Vue.js - The Progressive JavaScript Framework, 2014. letzter Zugriff am 04.04.2026.

[11] Figma, Inc. Figma - The Collaborative Interface Design Tool, 2016. letzter Zugriff am 04.04.2026.

[12] Google. Bigquery documentation, 2024. letzter Zugriff am 31.03.2026.

[13] Google. Google cloud documentation, 2024. letzter Zugriff am 31.03.2026.

[14] Google. Google play developer api, 2024. letzter Zugriff am 01.04.2026.

[15] Hello Again GmbH. Hello Again - Digitale Kundenbindung, 2017. letzter Zugriff am 04.04.2026.

[16] HubSpot Inc. Hubspot api documentation, 2024. letzter Zugriff am 31.03.2026.

[17] ISO. Iso/iec 9075 sql standard, 2016. letzter Zugriff am 31.03.2026.

[18] Peter Mell and Timothy Grance. The nist definition of cloud computing. *National Institute of Standards and Technology*, 2011. letzter Zugriff am 31.03.2026.

[19] Sergey Melnik et al. Dremel: Interactive analysis of web-scale datasets. *Proceedings of the VLDB Endowment*, 2010. letzter Zugriff am 31.03.2026.

[20] Pydantic Developers. Pydantic documentation, 2024. letzter Zugriff am 14.03.2026.

[21] Python Software Foundation. Python 3 documentation, 2024. letzter Zugriff am 24.02.2026.

[22] Sebastián Ramírez. Fastapi benchmarks and performance. *FastAPI Documentation*, 2024. letzter Zugriff am 14.03.2026.

[23] Sebastián Ramírez. Fastapi documentation, 2024. letzter Zugriff am 14.03.2026.

[24] Guido Van Rossum and Fred Drake. Python reference manual. *Python Software Foundation*, 2009. letzter Zugriff am 24.02.2026.

Abbildungsverzeichnis

Tabellenverzeichnis

Quellcodeverzeichnis

1 Grundstruktur eines FastAPI-Endpunkts 39
2 Definition des /health-checks Endpunkts 40
3 BigQuery-Abfrage im /health-checks Endpunkt 40
4 Aggregation von HubSpot-Tickets pro Kunde 41
5 Filterung von Asana-Tasks nach Section 41
6 Filterlogik im /health-checks Endpunkt 41
7 Extraktion verfügbarer Fehlercodes 42
8 Parametrisierte BigQuery-Abfrage im /client_details Endpunkt 42
9 Hilfsfunktion format_health_check 43
10 Statusänderungsfelder in der API-Antwort 43
11 BigQuery-Abfrage für Kunden-Tickets 44
12 Aufbereitung der Ticket-Daten 44
13 Filterung von Asana-Tasks nach Slug 45
14 Filterung von Nutzerkommentaren aus Asana Stories 45
15 Erstellung eines neuen Asana-Tasks 46
16 Ermittlung des aktuellen Nutzers über die Asana API 46
17 Initialisierung des BigQuery-Clients 47
18 Beispiel von Extraktion von Array-Feldern aus BigQuery-Ergebnissen . 48
19 Hilfsfunktion safe_get() für flexiblen Feldzugriff 48
20 Dateinamen der Snapshot-Dateien 49
21 Automatische Erstellung eines leeren Snapshots 49
22 Einlesen eines bestehenden Snapshots 50
23 Aktualisierung des Detailansicht-Snapshots 50
24 Berechnung von STATUS_CHANGED in der Übersicht 51
25 Feldweiser Vergleich in der Detailansicht 51

Anhang